Linear regression and convex optimization

Johannes Schäfers, Steffen Pablo Krieg, Ferdinand Nilius, Benedikt Schneider

Department of Physics,
University of Oslo, N-0316 Oslo,
Norway

Strategies used to train, select and solve different regression models can be accurately studied with simple models such as polynomial regression. Using Runge's function with Gaussian noise, we compared model performance while varying the sample size, the standard deviation of the noise, and the polynomial degree. We compared OLS to Ridge and Lasso, with K-fold cross validation and the bootstrap as model selection strategies, to study how and when penalization can lead to increases in model performance. We used the optimization algorithms (Stochastic) Gradient Descent, Momentum, RMSProp, AdaGrad, and Adam as solvers and compared them in terms of accuracy and convergence speed. The optimal polynomial degree, identified by the lowest MSE, is 11 for OLS at n = 150, whereas Ridge and Lasso find the optimal cross-validated MSE at polynomial degrees of 12 and 18, respectively. The results of this paper demonstrate that model selection, the tuning of parameters, and the use of resampling techniques play a decisive role in the quality of the generated models.

CONTENTS

I. Introduction 1

II. Methods and Algorithms 3
A. OLS and regularized regression 3
1. Singular Value Decomposition 3
2. Optimal solutions for Ordinary Least Squares 3
3. Extending the solution to Ridge 3
4. Lasso Regression 4
5. Scaling and centering of data 4
B. Measures of model quality 4
C. The Bootstrap resampling technique and Cross-validation 5
1. The Bootstrap 5
2. Cross-validation 5
3. MSE, bias and variance 5
D. Gradient Descent 6
1. Plain Gradient Descent 6
2. Obtaining the Gradient: Automatic vs. Analytical Differentiation 7
E. Optimization algorithms with dynamic learning rates 7
1. Momentum 7
2. AdaGrad 8
3. RMSprop 8
4. Adam 8
5. Numerical experiments 8
F. Stochastic Gradient Descent 9

III. Analysis of the results 9
A. Polynomial regression for the Runge function 9
B. Ridge regression for the Runge function 10
C. The Bootstrap resampling technique and Cross-validation 11
1. Train and test MSE of different models 11
2. The bias-variance decomposition 12
3. Cross-validation for OLS regression 14
4. Cross-validation for Ridge regression 14
D. Convergence Dynamics and Hyperparameter Sensitivity of Modern First-Order Optimizers 15
1. Comparison of different gradient descent methods 15
E. Lasso Regression and discussion of different gradient descent approaches 19
1. Discussion of OLS, *scikit-learn*-Lasso, Ridge and the self-written Implementation 19
2. Comparison of the Lasso code with *scikit-learn* 20
3. Confirming the results with JAX Automatic Differentiation Verification 20
F. Stochastic gradient descent in comparison with closed forms and full gradient descent 20
1. Comparison of different mini-batch sizes 20
2. Evaluating the fit quality 21
3. Dependence on the learning rate schedule 22
4. Final accuracy relative to closed-form-solutions 22
5. Limitations and Proposed Enhancements 23
G. Final comparison between OLS, Ridge and Lasso Regression with cross validation 23

IV. Conclusion 25

References 26

A. Table of abbreviations 28

B. Coefficients of the OLS model fitted to the Runge function 28

C. Gradient descent optimizer 28

D. Numerical Values for each epoch 30

E. Example: Lasso outperforms Ridge Regression 30

F. AI Declaration 30

I. INTRODUCTION

Fitting data is one of the most important topics when analyzing results, no matter whether they result from empirical experiments or from a new theory. When data do not follow an obvious correlation or the underlying law is unknown, guessing a correlation often pushes the given data in one direction. Furthermore, when data sets become larger, fitting the data analytically is often limited by sheer computational complexity. Iterative methods with optimization algorithms can be of use here. In the following, we present use cases for different regression models and optimization algorithms, compare them and analyze their strengths and weaknesses.

A polynomial function can be the best fit, as it is more unbiased than guessing correlations. However, using a polynomial can lead to problems. At first sight, it seems easy to fit training data using high-order polynomial features; the fit can reach most points nearly perfectly, reducing the mean squared error on the training data set. However, applying the fitted parameters to unseen test data will fail, as high-degree models are prone to overfitting of training-set fluctuations rather than learning the general underlying function. This phenomenon of overfitting is well demonstrated by Runge's function ($f(x) = \frac{1}{1+25x^2}$), where polynomial interpolation over equidistant points with higher-degree polynomials causes strong oscillations near the edges of the interval even without any additional noise. (Runge, 1901)

Fitting polynomial functions can be realized using Ordinary Least Squares Regression, which minimizes the sum of squared residuals with no penalization. It produces unbiased assumptions, which can, however, suffer when working with noisy or highly correlated data.(Hjorth-Jensen, 2026, sec. 3.3) When data are either highly correlated or noisy, introducing a penalty term, which can suppress unnecessary or correlated features, can be advantageous. Ridge regression adds a squared magnitude ($L2$) penalty that smoothly shrinks features towards zero, thereby preventing overfitting. (Hjorth-Jensen, 2026, sec. 3.8) If many features are given, but not all are useful, Lasso regression is the better option, as it is capable of shrinking parameters exactly to zero. It adds an absolute value ($L1$) penalty, which forces unnecessary features to zero. It therefore provides variable selection by completely suppressing features with low impact. (Hjorth-Jensen, 2026, sec. 3.9)

Often analytical solutions are not available or are computationally infeasible, especially for large-scale, nonlinear, or high-dimensional models. A fundamental algorithm for optimization problems is gradient descent (Hjorth-Jensen, 2026, sec. 4.3). The algorithm minimizes a loss function by moving iteratively in the direction of steepest descent. A full gradient descent evaluates the gradient over every single training sample before taking a step; thus, it converges towards the global minimum, when the appropriate stability conditions are fulfilled. As datasets are often too big to calculate a full gradient, stochastic gradient descent can be used. It splits the data into mini-batches and calculates the gradient for a small batch before doing an update. While this accelerates convergence, high gradient variance causes noisy updates that help escape local minima but hinder smooth convergence. (Hjorth-Jensen, 2026, sec. 4.7) In areas with a steeper curve in one direction than in others, SGD struggles to find the minimum as it is oscillating around it. Here momentum can help. It adds exponentially decaying fractions of the previous update vectors to build velocity in consistent directions and cancels out opposing directional steps, thereby smoothing oscillations. (Hjorth-Jensen, 2026, sec. 4.6) Last but not least, there are optimizers (AdaGrad, RMSProp, Adam). They all have one thing in common: eliminating the need for manually tweaking the learning rates for every parameter by automatically adjusting the step size based on previous gradients. (Hjorth-Jensen, 2026, sec. 4.9-4.11) AdaGrad scales the learning rates inversely to the total history of squared gradients. This can lead to a frozen learning rate as the step size goes to zero. (Duchi *et al.*, 2011) RMSProp solves the problem of premature stopping by using a rolling average of recent squared gradients instead of an ever-growing total sum. (Hinton, 2012) The standard optimizer is Adam; it combines the rolling gradient scaling of AdaGrad with the direction-stabilizing properties of Momentum. It tracks both where the gradient is pointing and how widely it fluctuates. (Kingma and Ba, 2015)

However, it can be hard to determine the quality of the fit when only limited test points are available. To evaluate the performance and to quantify the uncertainty without additional data points, validation methods such as the bootstrap and k-fold cross-validation are needed. The bootstrap approximates the sampling distribution of any statistic and (Hjorth-Jensen, 2026, sec. 2.11) primarily measures the statistical stability. The second option is *k-fold cross-validation*. This distributes the data into folds of equal size, where the model is tested on every fold once while the remaining folds are always used to train it. The bootstrap can also be used to perform a bias-variance decomposition (Efron, 1979; Hjorth-Jensen, 2026, sec. 2.11).

Using the methods presented, we present an introduction to the algorithms in section II. We also investigated how the quality of a fit using polynomial regression changes based on the sample size of the training data, the noise in the data, and the degree of polynomial used. Furthermore, the same properties will be explored for the regularized regression methods Ridge and Lasso. To solve these models we compared closed form solution to iterative methods such as Adam or RMSProp. Throughout this paper, we used the provided code as a tool to analyze different fitting methods, to compare the results of fitting, or to compare the speed of computation. Often only some of the code output will be presented. All Code can be found under: GitHub Repository – Project 1.

The primary goal of this project is to systematically evaluate how regularized regression models, resampling techniques, and adaptive optimization algorithms mitigate overfitting and numerical instability when fitting Runge's function across varying noise levels and sample sizes. This is summarized and answered in section IV.

The report concludes with an appendix, which also includes a declaration of large language model usage.

II. METHODS AND ALGORITHMS

In the following section, we introduce all relevant models and algorithms, including their mathematical structure and solution. We used all presented algorithms to solve one or more of the models. For our implementation of the respective algorithms, we relied on the Python packages *NumPy* (Harris *et al.*, 2020), *scikit-learn* (Pedregosa *et al.*, 2011) and *JAX* (Bradbury *et al.*, 2018).

A. OLS and regularized regression

Ordinary Least Squares regression models seek to minimize the mean squared residual, based on a single or a set of features. Therefore, when assuming that the data can be fitted using a linear model, the cost function for such a problem can be defined as(Hjorth-Jensen, 2026, sec. 3.3):

$$C_\theta = \frac{1}{n}\sum_{i=1}^{n}(y_i - \boldsymbol{x_i}^T\boldsymbol{\theta})^2. \tag{1}$$

Here $\boldsymbol{\theta}$ is the set of coefficients, while $\boldsymbol{x_i}$ and y_i represent the set of features and the actual values, respectively. If the individual data points are rewritten into a design matrix $\boldsymbol{X}$ and a vector containing the data to be fitted $\boldsymbol{y}$, the optimal solution to the problem can be written as (Hjorth-Jensen, 2026, sec. 3.3):

$$\boldsymbol{\theta_{opt}} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y} \tag{2}$$

1. Singular Value Decomposition

Finding the inverse of a matrix can be highly unstable. For OLS, this instability is worsened by the squaring of the condition number through the multiplication of $\boldsymbol{X}^T\boldsymbol{X}$. Factorization methods, such as QR and Cholesky decompositions, can help by either preventing the squared condition number or avoiding a direct matrix inversion. We implemented closed-form solutions for OLS and Ridge using singular value decomposition, which can be described as a factorization of a matrix into three different matrices (Hjorth-Jensen, 2026, sec. 1.18). This can be done for any matrix and can be described as follows:

$$\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T. \tag{3}$$

Here $\boldsymbol{U}$ and $\boldsymbol{V}$ are orthogonal matrices of size $n \times n$ and $p \times p$, respectively, while $\boldsymbol{\Sigma}$ is an $n \times p$ matrix that features only diagonal values, which are called singular values. (Hjorth-Jensen, 2026, sec. 1.18) We used the Python package *NumPy* (Harris *et al.*, 2020) in order to accurately compute the needed singular value decompositions.

2. Optimal solutions for Ordinary Least Squares

In order to find numerically stable solutions to the posed OLS problems, we use the singular value decomposition and the Moore-Penrose-pseudoinverse ($\boldsymbol{X}^+ = \boldsymbol{V}\boldsymbol{\Sigma}^+\boldsymbol{U}^T$) to rewrite equation 2 . This gives the following expression:

$$\boldsymbol{\theta_{opt}} = \boldsymbol{V}\boldsymbol{\Sigma}^{-2}\boldsymbol{V}^T\boldsymbol{V}\boldsymbol{\Sigma}^T\boldsymbol{U}^T\boldsymbol{y} = \boldsymbol{V}\boldsymbol{\Sigma}^+\boldsymbol{U}^T\boldsymbol{y} \tag{4}$$

For these equations $\boldsymbol{\Sigma}^+$ is a matrix computed by transposing $\boldsymbol{\Sigma}$ and taking the reciprocal of every non-zero singular value on the diagonal, while leaving the zeros untouched. (Hjorth-Jensen, 2026, sec. 1.19)

For plain OLS without the introduction of a regularization method or a resampling technique, we sampled 100 points from the Runge function with additional random Gaussian noise with a standard deviation of 0.1. The data was divided into a training set of 80 data points and a test set of 20 data points. Using the 80 data points, we fitted polynomial functions of varying degrees up to degree 15. Our expectation was to see an increase in model quality, measured by the error on the test set, with higher polynomial degrees until the model starts to overfit the training data. Furthermore, using a single degree as an example, we set out to study the dependence on the noise and the sample size. Here we expected to find that higher sample sizes lead to better models, while raising the underlying noise will raise the error.

3. Extending the solution to Ridge

Ridge regression is a regularization method aiming to decrease the variance of a model through the introduction of a penalty term. The goal of Ridge is therefore shifted in comparison to OLS: Instead of minimizing the MSE, it minimizes the MSE with the addition of a penalty term, which is based on the $L2$-norm of the parameters. (Hjorth-Jensen, 2026, sec. 3.8)

$$\min_{\boldsymbol{\theta}\in\mathbb{R}^p}\left\{\frac{1}{n}\left\|\boldsymbol{y} - X\boldsymbol{\theta}_{\text{Ridge}}\right\|_2^2 + \lambda\left\|\boldsymbol{\theta}_{\text{Ridge}}\right\|_2^2\right\} \tag{5}$$

Ridge regression has a closed-form solution resembling the closed-form solution of OLS up to an additional value on the diagonal of the matrix to be inverted. One additional bonus of Ridge regression is that the new matrix, which needs to be inverted, is better conditioned, as its eigenvalues are guaranteed to be larger than the penalty hyperparameter λ. In fact, it can be shown that the matrix is always invertible as long as $\lambda > 0$. The singular value decomposition can be used to further understand the results Ridge regression produces. Through the decomposition of the design matrix $\boldsymbol{X}$, the closed-form solution can be rewritten as (Hjorth-Jensen, 2026, sec. 1.22):

$$\boldsymbol{\theta}_{\text{Ridge}} = \sum_{i=0}^{r-1} \frac{\sigma_i}{\sigma_i^2 + n\lambda} (\boldsymbol{u_i}^T \boldsymbol{y}) \boldsymbol{v_i}. \tag{6}$$

[1] Where $\boldsymbol{u_i}$ and $\boldsymbol{v_i}$ are the individual columns of the matrices $\boldsymbol{U}$ and $\boldsymbol{V}$, respectively. This again shows a form similar to the solution for OLS regression, when the SVD is used. The key difference between the two is that Ridge strongly shrinks the directions with low singular values through the factor $\frac{\sigma_i}{\sigma_i^2+\lambda}$, while leaving directions with larger singular values essentially untouched, whereas OLS treats every direction in the same way. (Hjorth-Jensen, 2026, sec. 1.22)

For Ridge regression, we set out to study the dependence of the error on both the polynomial degree and the hyperparameter λ. As we expected Ridge to be less prone to overfitting, we used polynomials of up to degree 30, with values of λ ranging between 10^{-5} and 10^3.

4. Lasso Regression

Like Ridge regression, Lasso provides a regularized linear regression model by penalizing the model coefficients. In contrast to Ridge, however, Lasso adds the L1-norm of the parameters to the cost function rather than the L2-norm. The cost function can therefore be written as (Hjorth-Jensen, 2026, sec. 3.9) (Tibshirani, 1996):

$$C(\boldsymbol{\theta}) = \frac{1}{n} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}_{\text{Lasso}}\|_2^2 + \lambda \|\boldsymbol{\theta}_{\text{Lasso}}\|_1 \tag{7}$$

In comparison to OLS and Ridge, Lasso does not have a closed-form solution to find the optimal parameters $\boldsymbol{\theta}$. However, Lasso's optimization problem is still convex in nature; therefore, it has a global minimum and can be solved using iterative procedures (Hjorth-Jensen, 2026, sec. 3.9). For a discussion of methods which can be used to solve Lasso regression, refer to the sections II.D.1, II.F, and II.E.

One important property of Lasso is the natural feature selection embedded in the penalization using the L1-norm. This is caused by the fact that Lasso only preserves features whose correlation with the residual is large enough to pay the price placed upon them through the penalty λ.

$$|\frac{2}{n}\boldsymbol{x_j}\boldsymbol{r}| \leq \lambda \,\text{if}\, \hat{\theta_j} = 0 \tag{8}$$

For further details on this, refer to (Hjorth-Jensen, 2026, sec. 3.9) and (Tibshirani, 1996).

[1] Note that (Hjorth-Jensen, 2026) uses λ instead of $n\lambda$, which is a difference in convention. Here we included $n\lambda$ to make the notation consistent.

5. Scaling and centering of data

The intercept of the model contains the value a model shall predict if every one of its coefficients is zero. In linear regression models, this is typically introduced through a column of ones in the design matrix. Usually, this coefficient should not be shrunk by penalization such as Ridge or Lasso regression apply it, as we want our model to shift with the underlying target, no matter the origin of said target. An elegant solution to this problem is to center both the target values in the vector $\boldsymbol{y}$ and all columns in the design matrix $\boldsymbol{X}$. The result of this centering is an intercept coefficient, which is guaranteed to be zero, as both the target and the predictors move to the origin.

The shrinkage that Ridge and Lasso apply through their hyperparameter λ treats all coefficients alike. However, the features on which each of the coefficients depends might have different units and scales. This can therefore result in differences in magnitude between the features, resulting in differences in the shrinkage strength of features. This can be resolved through scaling of all features alike, thereby creating a unified scale on which the individual features are measured. Throughout this paper, we scaled all features to have unit variance, which can be achieved through a division of every column by its own standard deviation. Furthermore, we centered both the target vectors $\boldsymbol{y}$ and the columns of the design matrix $\boldsymbol{X}$ by subtracting the mean value of each respective column from the values. (Hjorth-Jensen, 2026, sec. 3.13.) This method does not confine the values to any particular range. However, we chose it, because it provides robustness to scaling, good interpretability, as all coefficients now have unit variance, and removal of the intercept through centering.

$$\boldsymbol{y}_{\text{centered}} = \boldsymbol{y} - \frac{1}{n}\sum_{i=0}^{n-1} \boldsymbol{y_i} \tag{9}$$

$$\boldsymbol{x_{i,\,\text{scaled}}} = \frac{\boldsymbol{x_i} - \frac{1}{n}\sum_{j=0}^{n-1} \boldsymbol{x_{j,i}}}{\sigma_i} \tag{10}$$

Equations 9 and 10 show the mathematical formulas for the scaling and centering of the vector $\boldsymbol{y}$ and the individual columns of the design matrix $\boldsymbol{x_i}$, where σ_i is the standard deviation of column i of the design matrix.

This method of applying scaling and centering is equal to the 'StandardScaler' function featured in the Python package *scikit-learn* (Pedregosa *et al.*, 2011), which we also used in parts of the code for convenience.

B. Measures of model quality

In the following, we mainly assessed the predictions made by all models based on their mean squared error

(MSE) and the R^2 value on a test data set, which we did not include in the data we used to train the model. For this purpose, we define the MSE as:

$$\text{MSE} = \frac{1}{n} \cdot \sum_{i=0}^{n-1} (y_i - y_{pred})^2, \tag{11}$$

where y_{pred} is the value predicted by the model, while y_i is the actual value. This quantity's relevancy is given through the design of OLS itself, which optimizes the MSE based on the given training sample. (Hjorth-Jensen, 2026, sec. 3.5.)
As the mean squared error scales with the magnitude of the target, it can be helpful to define additional quantities which can represent the quality of the fit independently of the scale of the target. Throughout this paper we used the R^2 value, which is defined as:

$$R^2 = 1 - \frac{\sum_{i=0}^{n-1} (y_i - y_{pred})^2}{\sum_{i=0}^{n-1} (y_i - y_{mean})^2}. \tag{12}$$

The R^2 value therefore usually ranges between 1 and 0, 1 representing a perfect fit and 0 representing a fit as good as a constant prediction of the mean. Negative values can be achieved when a model is a worse predictor of the values than the mean value of the target data, thereby indicating a poorly fitting model. (Hjorth-Jensen, 2026, sec. 3.5.)

C. The Bootstrap resampling technique and Cross-validation

The properties of a model can be described by values like the MSE. But this only characterizes the properties of one specific model; it does not take into account that a model may produce different results depending on the data on which it has been trained and tested upon. This is where the bootstrap resampling technique and cross validation help. These methods are used to describe the different behaviors of different models, trained and tested on different parts of the data. They use different approaches to achieve this.

1. The Bootstrap

When using the bootstrap, the data is first split into train and test data, just as when characterizing a single model. To train the model on different sets, the train data are then shuffled. This shuffling is equivalent to sampling with replacement from both the x and y training sets (of course in the same way for x and y so that the points still match) and storing the sampled points in the new data set. The model is then trained on this new dataset and the predicted y-values for the test set are stored. The test set is never shuffled to ensure comparability for the bias variance decomposition. (Hjorth-Jensen, 2026, sec. 2.11.)

2. Cross-validation

For cross-validation, the different data sets are produced differently. The data are distributed into folds of equal size, where the model is tested on every fold once while the remaining folds are always used to train it. This can only investigate the MSE and not bias and variance because the test set is never the same.

3. MSE, bias and variance

For both of the methods, the mean MSE can be calculated as:

$$\mathbb{E}\left[(\boldsymbol{y} - \tilde{\boldsymbol{y}})^2\right] \tag{13}$$

where the mean $\mathbb{E}$ means the mean over the folds or bootstraps. For the bootstrap, the MSE can be decomposed into a bias, a variance and a noise term:

$$\mathbb{E}\left[(\boldsymbol{y} - \tilde{\boldsymbol{y}})^2\right] = \text{Bias}^2[\tilde{y}] + \text{var}[\tilde{y}] + \sigma^2, \tag{14}$$

with

$$\begin{aligned} \text{Bias}^2[\tilde{y}] &= \mathbb{E}\left[\left(\boldsymbol{f} - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2\right], \\ \text{var}[\tilde{y}] &= \mathbb{E}\left[\left(\tilde{\boldsymbol{y}} - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2\right] \\ &= \frac{1}{n} \sum_i \left(\tilde{y}_i - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2. \end{aligned} \tag{15}$$

Proof:
We already know that

$$var\left[\tilde{\boldsymbol{y}}\right] = \mathbb{E}\left[\tilde{\boldsymbol{y}}^2\right] - \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]^2$$

and therefore

$$\mathbb{E}\left[\tilde{\boldsymbol{y}}^2\right] = var\left[\tilde{\boldsymbol{y}}\right] + \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]^2.$$

We assume that

$$\boldsymbol{y} = \boldsymbol{f} + \epsilon.$$

If we substitute both of these expressions into our equation, expand the parentheses, and simplify, we get the following expression:

$$\begin{aligned} &\mathbb{E}\left[\boldsymbol{y}^2\right] - 2\mathbb{E}\left[\boldsymbol{y}\tilde{\boldsymbol{y}}\right] + \mathbb{E}\left[\tilde{\boldsymbol{y}}^2\right] \\ &= var\left[\tilde{\boldsymbol{y}}\right] + \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]^2 + \mathbb{E}\left[\boldsymbol{f}^2\right] \\ &\quad + 2\mathbb{E}\left[\boldsymbol{f}\epsilon\right] + \mathbb{E}\left[\epsilon^2\right] - 2\mathbb{E}\left[\tilde{\boldsymbol{y}}\epsilon\right] \\ &\quad - 2\mathbb{E}\left[\tilde{\boldsymbol{y}}\boldsymbol{f}\right]. \end{aligned}$$

We also assume that our noise ϵ has a zero mean and is also independent of $\tilde{\boldsymbol{y}}$. Furthermore $\boldsymbol{f}$ is non-stochastic and can therefore be pulled out of the expectation value operator. Therefore, two of the terms in this expression are equal to zero.

$$\begin{aligned}\mathbb{E}\left[\tilde{\boldsymbol{y}}\epsilon\right] &= \mathbb{E}\left[\epsilon\right]\mathbb{E}\left[\tilde{\boldsymbol{y}}\right] = 0,\\ \mathbb{E}\left[\boldsymbol{f}\epsilon\right] &= \boldsymbol{f}\mathbb{E}\left[\epsilon\right] = 0.\end{aligned}$$

By definition, the expectation value of ϵ^2 is equal to the variance of the noise σ^2.

Our expression now reads:

$$\begin{aligned}&var\left[\tilde{\boldsymbol{y}}\right] + \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]^2 + \mathbb{E}\left[\boldsymbol{f}^2\right]\\ &+ 2\mathbb{E}\left[\boldsymbol{f}\epsilon\right] + \mathbb{E}\left[\epsilon^2\right] - 2\mathbb{E}\left[\tilde{\boldsymbol{y}}\epsilon\right]\\ &- 2\mathbb{E}\left[\tilde{\boldsymbol{y}}\boldsymbol{f}\right]\\ =\ &var\left[\tilde{\boldsymbol{y}}\right] + \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]^2 + \mathbb{E}\left[\boldsymbol{f}^2\right]\\ &+ \sigma^2 - 2\boldsymbol{f}\mathbb{E}\left[\tilde{\boldsymbol{y}}\right].\end{aligned}$$

This can be further simplified and rearranged to find our final expression:

$$\begin{aligned}&var\left[\tilde{\boldsymbol{y}}\right] + \sigma^2 + \mathbb{E}\left[\left(\boldsymbol{f} - \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]\right)^2\right]\\ =\ &\frac{1}{n}\sum_i \left(\tilde{\boldsymbol{y}} - \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]\right)^2\\ &+ \frac{1}{n}\sum_i \left(\boldsymbol{f_i} - \mathbb{E}\left[\tilde{\boldsymbol{y}}\right]\right)^2 + \sigma^2.\end{aligned}$$

This is called the bias-variance decomposition. The variance measures how much the predictions of the different models vary from each other while the bias measures the global offset of the data from the exact value it should predict. The MSE is the MSE we already know, except that it is averaged over many models to gain a mean expected MSE. Cross-validation and the bootstrap can be used to find the average MSE for different values of λ for Ridge and Lasso regression or for different degrees for linear OLS regression.

D. Gradient Descent

1. Plain Gradient Descent

As a first optimizer, we considered gradient descent (often also called plain gradient descent or full gradient descent to distinguish it from other gradient descent methods). The algorithm can be motivated by Newton's method, which finds the minimum of the cost function $C(\boldsymbol{\theta})$ by finding the root of its gradient $\nabla_\theta C(\boldsymbol{\theta})$ with the following iteration:(Hjorth-Jensen, 2026, sec. 4.2)

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - H^{-1}(\boldsymbol{\theta}_t)\nabla C(\boldsymbol{\theta}_t). \tag{16}$$

Here, $H^{-1}(\boldsymbol{\theta}_t)$ denotes the inverse Hessian of the cost function. To avoid the costly computation of the inverse Hessian at every iteration, gradient descent replaces it with $\gamma\boldsymbol{I}$, where $\gamma > 0$ is a small constant learning rate (what *small* means here is discussed below)(Hjorth-Jensen, 2026, sec. 4.3):

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma\nabla C(\boldsymbol{\theta}_t). \tag{17}$$

The gradient descent algorithm is given in the following pseudo-code:

Algorithm 1: Gradient Descent

Require: Cost function $C(\boldsymbol{\theta})$, initial parameters $\boldsymbol{\theta}_0$, learning rate $\gamma > 0$, tolerance $\delta > 0$, maximum number of iterations T
Ensure: Approximate minimizer $\boldsymbol{\theta}^*$

```
t ← 0
while t < T do
    g_t ← ∇C(θ_t)
    if ‖g_t‖ < δ then
        break                          ▷ Converged
    end if
    θ_{t+1} ← θ_t − γ g_t
    t ← t + 1
end while
return θ* ← θ_t
```

Using the norm of the gradient as a convergence criterion works well for convex problems and is therefore suitable for this work. However, we focused on the actual distance to the minimum, $\left\|\boldsymbol{\theta}_t - \hat{\boldsymbol{\theta}}\right\|$, where $\hat{\boldsymbol{\theta}}$ is the closed-form solution of either OLS or Ridge regression.

We expected plain gradient descent to be sensitive to the choice of the learning rate γ and therefore needed well-founded criteria for choosing it. Focusing on OLS and Ridge regression, (Hjorth-Jensen, 2026, sec. 4.5) shows that the eigenvalues of the Hessian provide such a criterion: gradient descent does not diverge as long as

$$0 < \gamma < \gamma_{\max} = \frac{2}{\lambda_{\max}}, \tag{18}$$

where $\lambda_{\max}$ is the largest eigenvalue of the Hessian. Moreover, the theoretically optimal learning rate for a quadratic problem is

$$\gamma^* = \frac{2}{\lambda_{\max} + \lambda_{\min}}, \tag{19}$$

where $\lambda_{\min}$ denotes the smallest nonzero eigenvalue.

Lastly, the condition number for the OLS regression is given by

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} \tag{20}$$

For Ridge regression, the condition number in eq. (20) adjusts to:

$$\kappa_{Ridge} = \frac{\lambda_{\max} + 2\lambda_{Ridge}}{\lambda_{\min} + 2\lambda_{Ridge}} \tag{21}$$

The number of iterations until convergence of plain gradient descent scales linearly with the condition number(Hjorth-Jensen, 2026, sec. 4.4.). As we used $\lambda_{ridge} > 0$ for Ridge, we expected the number of iterations until convergence to always be smaller than for OLS.

2. Obtaining the Gradient: Automatic vs. Analytical Differentiation

Gradient descent requires the gradient to be computed at every iteration. In the cases considered here, we have the luxury of an explicit analytical expression for the gradient. For Ridge regression, it reads

$$\nabla C_{\text{Ridge}}(\boldsymbol{\theta}) = \frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}) + 2\lambda\boldsymbol{\theta}. \tag{22}$$

For $\lambda \to 0$, eq. (22) reduces to the gradient of OLS.

Looking ahead, gradient descent will also be applied to far more complex cost functions for which no simple closed-form expression of the gradient is available. This leaves three ways of computing the gradient:

1. **Symbolic differentiation:** Computer algebra systems such as *SymPy* or *Mathematica* derive analytical, but in general computationally expensive, expressions for the gradient. Less expensive methods may therefore be preferable.

2. **Numerical differentiation:** The derivative is approximated by the central difference

$$f'(x) = Df(x) \approx \frac{f(x+h) - f(x-h)}{2h}. \tag{23}$$

   The error of eq. (23) scales as

$$E(h) \approx \frac{h^2}{6}|f'''(x)| + \frac{\varepsilon_M\,|f(x)|}{h}, \tag{24}$$

   where ε_M denotes the machine precision. Since we used double-precision floating-point numbers, $\varepsilon_M \sim 10^{-16}$, and the error of the gradient is at best of order $E(h) \sim 10^{-11}$, i.e. about five orders of magnitude larger than machine precision (Hjorth-Jensen, 2026).

3. **Automatic differentiation:** Automatic differentiation computes the gradient at low computational cost and exactly up to floating-point round-off. Automatic differentiation libraries, such as *JAX* (Bradbury *et al.*, 2018), decompose the cost function into elementary operations and evaluate the gradient by repeated application of the chain rule.

To verify our use of automatic differentiation, we compared the analytical gradients of the OLS and Ridge cost functions with the gradients obtained by automatic differentiation. We expected the two to agree up to machine precision, i.e. to a difference of order 10^{-16}.

To quantify the cost of automatic differentiation relative to one evaluation of the cost function, we counted floating-point operations (flops), treating every addition, multiplication and exponentiation (including subtractions and divisions) as one flop. For a design matrix $X \in \mathbb{R}^{n\times p}$ and the residual $r = X\theta - y$, one evaluation of the OLS cost function requires $2np + 2n$ flops: $n(2p-1)$ for $X\theta$, n for the subtraction, n for the squares, $n-1$ for the sum and one for the factor $1/n$. The penalty term of Ridge regression adds $2p+1$ flops. With the constants $2/n$ and 2λ computed once, the analytical gradient in eq. (22) requires $4np$ flops for OLS and $4np+2p$ flops for Ridge regression. Reverse-mode automatic differentiation, as used by JAX, first evaluates the cost function while storing all intermediate results (forward pass) and then applies the chain rule backwards through the same elementary operations (backward pass). The backward pass mirrors the forward pass and requires $2np+2n-p+1$ flops for OLS, so that the complete gradient costs $4np+4n-p+1$ flops for OLS and $4np+4n+4p+3$ flops for Ridge regression. For the $n = 100$ data points and $p = 5$ features used in section III.D, this amounts to 2396 and 2423 flops, compared with 1200 and 1211 flops for one evaluation of the respective cost function and 2000 and 2010 flops for the analytical gradient. Automatic differentiation therefore computes the complete gradient at about twice the cost of a single evaluation of the cost function, independently of the number of parameters (Baydin *et al.*, 2018). The central difference in eq. (23), in contrast, requires $2p$ evaluations of the cost function, i.e. 12 020 flops for OLS. The remaining overhead of about 20% compared with the analytical gradient stems from applying the chain rule element by element; in return, the forward pass yields the value of the cost function at no additional cost.

As gradient descent does not care where the gradient comes from, we implement and verify the functionality of both, but restrict ourselves to using the analytical gradient throughout the analysis.

E. Optimization algorithms with dynamic learning rates

All of the following adaptations of gradient descent aim to modify the optimization step $\boldsymbol{\theta}_t \to \boldsymbol{\theta}_{t+1}$ such that the effective learning rate changes dynamically based on previous steps. Below, we briefly introduce each optimization step and point to further resources on the matter.

1. Momentum

Momentum optimization updates $\boldsymbol{\theta}$ not by the current gradient but by an exponentially weighted sum of all previously encountered gradients, i.e. the optimization step

becomes:

$$\boldsymbol{v}_{t+1} = \beta \boldsymbol{v}_t + \gamma \nabla C\left(\boldsymbol{\theta}_t\right) \tag{25}$$
$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \boldsymbol{v}_{t+1} \tag{26}$$

Here, the *momentum parameter* $\beta \in [0, 1)$ (typically $\beta \approx 0.9$) is introduced. For slowly varying gradients, the momentum parameter effectively increases the learning rate, $\gamma \to \frac{\gamma}{1-\beta}$, and thereby accelerates gradient descent. On the other hand, momentum dampens oscillations: in oscillating regions, $\boldsymbol{v}_{t+1}$ in eq. (25) sums up gradients with alternating signs, which effectively cancel each other out. This leads to the following effect: the number of iterations required for convergence now scales with $\propto \sqrt{\kappa}$ (compared to $\propto \kappa$ for plain GD). For further insight, refer to (Hjorth-Jensen, 2026, sec. 4.6).

2. AdaGrad

AdaGrad decreases the step size in directions with large gradients and increases it in directions with small gradients. Thereby, each update is scaled by the inverse square root of the sum of all previous squared gradients. Element-wise:

$$r_{t+1,j} = r_{t,j} + g_{t,j} \cdot g_{t,j} \tag{27}$$
$$\theta_{t+1,j} = \theta_{t,j} - \frac{\gamma}{\sqrt{\varepsilon + r_{t+1,j}}} g_{t,j} \tag{28}$$

The small constant ε is added for numerical stability, avoiding divisions by zero.

However, AdaGrad has a limitation: since $\boldsymbol{r}_t$ increases monotonically, the effective learning rate in eq. (28) decreases monotonically. This increases the chance that the optimization comes to a halt before the minimum is reached. For more information, see (Hjorth-Jensen, 2026, sec. 4.9).

3. RMSprop

RMSprop mitigates this limitation of AdaGrad by scaling the update step with an exponentially weighted sum of the squared gradients. Element-wise:

$$v_{t+1,j} = \rho \cdot v_{t,j} + (1-\rho)\left(g_{t,j} \cdot g_{t,j}\right) \tag{29}$$
$$\theta_{t+1,j} = \theta_{t,j} - \frac{\gamma}{\sqrt{\varepsilon + v_{t+1,j}}} g_{t,j} \tag{30}$$

This introduces another hyperparameter, $\rho \in (0, 1)$, which is typically chosen as 0.9 or 0.99. The idea is that a squared gradient which enters at step s with weight $(1-\rho)$ has, $k = t - s$ steps later, only the reduced weight $(1-\rho)\rho^k$. That is, gradients further in the past have an increasingly smaller impact on the current learning rate. Avoiding this long reach of past gradients comes at a different cost: once RMSprop is close to a local minimum, it is typically unable to settle, especially for convex polynomial problems, and instead oscillates around the minimum with a step size of order γ. We expected to reproduce this behavior in our later analyses.

Refer to (Hjorth-Jensen, 2026, sec. 4.10) for a further explanation of how the finite memory of RMSprop prevents it from settling.

4. Adam

Adam, as proposed in (Kingma and Ba, 2015), combines the advantages of AdaGrad (section II.E.2) and RMSprop (section II.E.3). All vector operations are to be understood element-wise:

$$\boldsymbol{g}_t = \nabla C(\boldsymbol{\theta}_{t-1}) \tag{31}$$
$$\boldsymbol{m}_t = \beta_1 \cdot \boldsymbol{m}_{t-1} + (1-\beta_1) \cdot \boldsymbol{g}_t \tag{32}$$
$$\boldsymbol{v}_t = \beta_2 \cdot \boldsymbol{v}_{t-1} + (1-\beta_2) \cdot (\boldsymbol{g}_t)^2 \tag{33}$$
$$\hat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1-\beta_1^t} \tag{34}$$
$$\hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_2^t} \tag{35}$$
$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \gamma \cdot \frac{\hat{\boldsymbol{m}}_t}{\sqrt{\hat{\boldsymbol{v}}_t} + \varepsilon} \tag{36}$$

Here, $\boldsymbol{m}_t$ is an exponentially weighted average of past gradients and thus plays the role of the momentum term (section II.E.1), while $\boldsymbol{v}_t$ corresponds to the squared-gradient average of RMSprop. Since both are initialized as $\boldsymbol{m}_0 = \boldsymbol{v}_0 = \boldsymbol{0}$, they are biased towards zero during the first iterations. The bias-corrected estimates $\hat{\boldsymbol{m}}_t$ and $\hat{\boldsymbol{v}}_t$ compensate for this. The two new hyperparameters are typically chosen as $\beta_1 = 0.9$ and $\beta_2 = 0.999$, as empirically determined by (Kingma and Ba, 2015).

5. Numerical experiments

We implemented all four algorithms, and compared their convergence behavior with that of plain gradient descent.

Throughout all experiments in this section, we used the following settings.

$$\begin{aligned} \text{polynomial degree (OLS/Ridge)} &= 5 \\ \lambda_{\text{Ridge}} &= 0.1 \\ \beta &= 0.9 \\ \rho &= 0.99 \\ \beta_1 &= 0.9 \\ \beta_2 &= 0.999 \\ \varepsilon &= 10^{-8} \end{aligned}$$

The initial parameters $\boldsymbol{\theta}_0$ are drawn randomly, using the seed 2026. We still restricted ourselves to full gradient descent; we did not yet use stochastic mini-batches.

For convergence analysis, we plotted the Euclidean distance to the closed-form OLS and Ridge solutions, $\left\|\boldsymbol{\theta}_t - \hat{\boldsymbol{\theta}}\right\|_2$, against the number of iterations. We used the convergence criterion $\left\|\boldsymbol{\theta}_t - \hat{\boldsymbol{\theta}}\right\|_2 \leq 10^{-6}$ and consider an algorithm to not have converged if this criterion was not met within 20 000 iterations. We examined the learning rates $\gamma \in \{0.1, 0.5, 0.9, 0.99\} \cdot \gamma_{\max}$ and $\gamma = \gamma^*$, where $\gamma_{\max}$ and γ^* are the maximum and optimal learning rates of plain gradient descent as calculated following eqs. (18) to (19).

Using the same criterion, we additionally determined the number of iterations until convergence for a wider range of learning rates, using 50 logarithmically spaced values of $\gamma \in \left[10^{-3}, 10^{1}\right]$, for both OLS and Ridge. This allows us to evaluate how sensitive each optimizer is to the choice of the learning rate γ.

We expected plain gradient descent to be the most sensitive to the choice of learning rate: it diverges for $\gamma > \gamma_{\max}$, while significantly smaller learning rates lead to slow convergence, so that the algorithm reaches the iteration limit before the convergence criterion is met. As the condition number shrinks with $\lambda > 0$ the number of iteration to reach convergence will be always lower for Ridge in comparison to OLS.

At the other end of the spectrum, we expected Adam to be the least sensitive to the learning rate and to perform best over a wide range, as it combines the benefits of RMSprop and AdaGrad.

We expected momentum to converge in a similar range of learning rates as plain gradient descent, but to require fewer iterations throughout.

Finally, we expected to observe the oscillations of RMSprop around the minimum, with an amplitude of the order of the learning rate, and its resulting inability to settle.

F. Stochastic Gradient Descent

For large models with large sample sizes and possibly a large number of parameters, the computational complexity of evaluating a full gradient, as done in plain GD, can be the limiting factor. This warrants the introduction of a different step function, which is easier to compute, especially for larger data sets. One possible remedy of this problem is to introduce stochasticity by evaluating the gradient based only upon a subset of the data, usually called a minibatch B_i. Using this stochastic approach in a plain GD model leads to stochastic gradient descent (SGD), whose update is defined as(Hjorth-Jensen, 2026, sec. 4.7.):

$$\boldsymbol{\theta_{t+1}} = \boldsymbol{\theta_t} - \gamma_t \boldsymbol{g_{B_t}}(\boldsymbol{\theta_t}) = \boldsymbol{\theta_t} - \frac{\gamma_t}{M} \sum_{i \in B_t} \boldsymbol{\nabla} c_i(\boldsymbol{\theta_t}). \quad (37)$$

Here, B_t denotes the fresh minibatch used as the basis for the next update, while M is the size of the individual minibatches. Usually, a number of epochs is set as a possible stopping condition, where a single epoch is defined as a pass through all minibatches. The advantages of SGD are twofold: firstly, it reduces the computational complexity of a single step, thereby taking more steps in comparison with GD. Secondly, SGD introduces stochasticity and therefore randomness, which can be advantageous at points where plain GD might get stuck. This same noise also introduces SGD's disadvantage: if a minimum is found, SGD does not stop, but rather keeps jumping around. (Hjorth-Jensen, 2026, sec. 4.7.)

III. ANALYSIS OF THE RESULTS

A. Polynomial regression for the Runge function

Using the Runge function, we set out to study the dependence of the quality of polynomial regression models on their polynomial degree. We scaled the data according to equations 10 and 9 and calculated both the mean values and the standard deviation based on the training data set in order to avoid spillage from the test set.

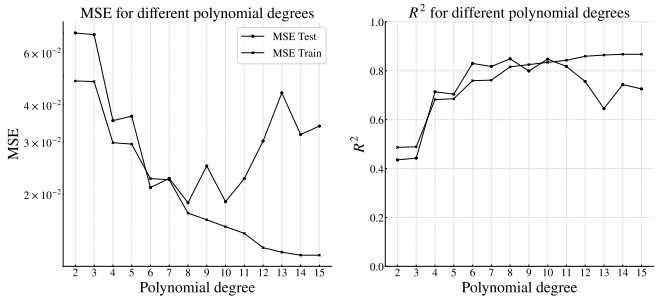


FIG. 1: Comparison of training and test MSE (left) and R^2 (right) as a function of polynomial degree for OLS on the Runge function with Gaussian noise of standard deviation 0.1. 100 sample points were divided into 80 training and 20 test points.

Figure 1 shows both the MSE (left) and the R^2 (right) for the training and test data sets. It can be seen that up to the optimal degree of 8 (here identified by the lowest MSE on the test data set), both the test error and the training error decrease near monotonically. The error in the test set stays mostly flat throughout the models using polynomials of degrees 6 to 11. After degree 11, a difference in the behavior of the graph of training and test error arises. While the error on the training data set keeps

steadily decreasing, the error on the test data set starts to increase. The first cause of this is the Runge function itself, which is famously hard to fit using higher-order polynomials (Runge, 1901), as the polynomials start oscillating between the training points, especially near the boundaries of the data. Furthermore, the model also starts overfitting the noise in the training data, which results in a further increase in test error. This can be more accurately seen when plotting the corresponding functions for the optimal degree-8 polynomial, as well as the function for the degree-15 polynomial as an example.

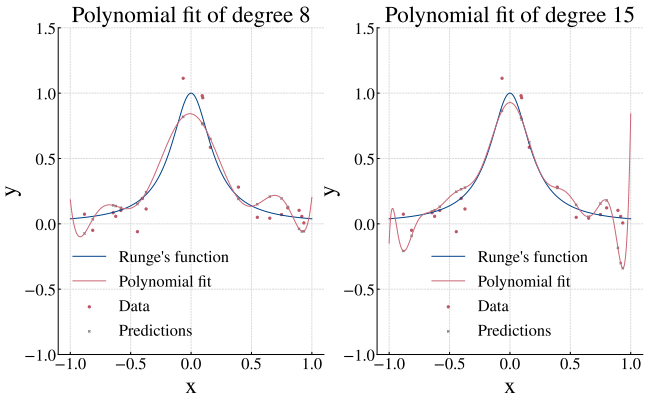


FIG. 2: The Runge function plotted against the approximations using the optimal degree 8 polynomial and the degree 15 polynomial approximation.

When viewing the resulting polynomial functions for the degree-8 polynomial and the polynomial of degree 15, the overfitting behavior becomes more obvious. This can be seen in Figure 2, where the degree-8 and degree-15 polynomials are plotted against the test data and the underlying Runge function. While the degree-15 polynomial can fit the data points close to the origin and the training points quite well, it shows strong oscillations at the boundaries, as we previously predicted. In comparison, the degree-8 polynomial may show a slightly worse fit at the origin. However, since this model does not overfit the training data as heavily and it does not have the large oscillatory effects caused by the higher-degree polynomial terms, the model provides overall better predictions. This therefore explains why a minimum in the MSE and a maximum in the R^2 value are seen at polynomials of degree 8 in Figure 1.
For the calculated coefficients of all polynomial OLS models (rounded to two decimal places) and a link to the plots of all computed coefficients, refer to Table VI in the appendix.

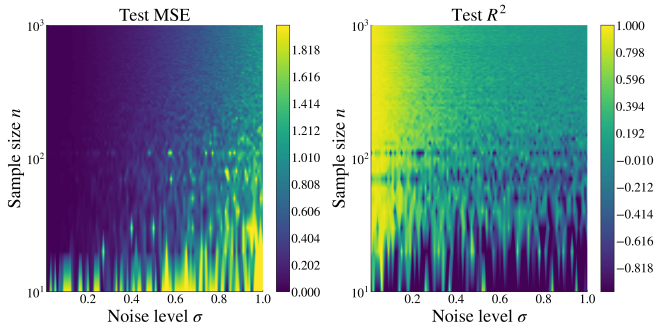


FIG. 3: Test-MSE (left) and Test-R^2 (right) for a polynomial of degree 8 fitted to the Runge function based on the number of samples and the standard deviation of the added noise. The MSE was cut off for values over 2, the y-axis is logarithmically scaled.

Figure 3 shows both the mean squared error and the R^2 on the test sets for different sample sizes and noise levels. We performed this fit using a polynomial degree of 8, which we previously identified to be optimal for a sample size of 100 data points, with normally distributed noise of standard deviation 0.1. As the MSE increases strongly when the level of the added Gaussian noise increases, the Figure is cut off at a value of 2. It can be seen that for low noise levels and high sample sizes, the model achieves a near-perfect fit, defined as having an MSE approaching zero and an R^2 approaching one. This is a result of a non-overfitting model, which can reproduce the Runge function to an accurate degree. It is also of note that even for low noise levels, a sample size of only 10 points, split into 8 training points and 2 test points, is too low to produce any accurate fit. This is because of the high level of overfitting created by fitting a polynomial of degree 8 on only 8 data points and leads to samples with very high MSE values even at low noise levels. Furthermore, it can be seen that even for large sample sizes, the R^2 of the model approaches zero for high noise values. Here, the model does not have any higher predictive power than a constant predictor of the mean value. It is of note that the polynomial of degree 8 is not optimal for all points on this grid, as higher polynomial degrees might lead to better results on bigger sample sizes, while lower-degree polynomials might create a better result for low sample sizes. This is exemplarily explored in section III.C.

B. Ridge regression for the Runge function

The previous section is concerned with the optimization of the degree of the polynomial used to fit the data. For OLS, the optimal polynomial degree is 8, with higher degree polynomials introducing oscillatory and overfitting effects. The introduction of Ridge regression instead of OLS introduces another hyperparameter (the penalty parameter λ) into our model. Similarly to the polynomial

degree, this parameter needs to be accurately selected and tuned in order to make full use of Ridge's predictive powers.

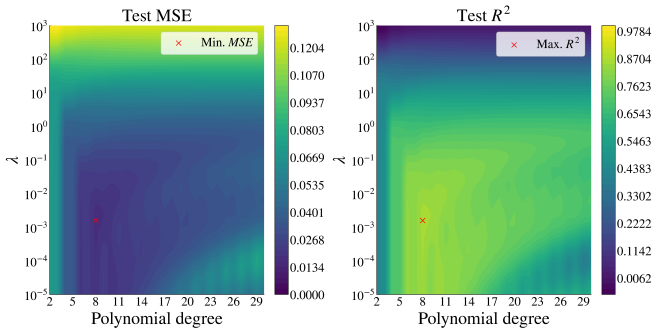


FIG. 4: Test-MSE (left) and Test-R^2 (right) for polynomial degrees between 2 and 30 and λ values between 10^{-5} and 10^3.

In order to find the optimal parameters for Ridge, we computed the model on a two dimensional grid, with λ values ranging from 10^{-5} to 10^3 and polynomials from degree 2 up to degree 30. Figure 4 shows the result of this parameter space exploration, plotting both the MSE and the R^2 value as a function of the polynomial degree and the hyperparameter λ. The optimal values for the hyperparameter λ and the degree of polynomials used are again defined as the values for which the model produces the smallest MSE on the test set. Similar to OLS, Ridge regression also shows an optimal fit at a polynomial degree of 8, while the optimal value for the hyperparameter is $\lambda \approx 0.016$. Through the contours drawn inside the Figure, it can be seen that Ridge regression can allow for higher degrees of polynomials to be used with similar model performance, as compared to OLS. This can be made obvious by tracing one of the contour lines originating from $\lambda \approx 0$ and following it to its point furthest right in the plot. As the MSE values along this contour are at a similar level, the quality of the models can be assumed to be similar. For example, the contour line starting at degree 14 also reaches some degree-16 and degree-17 polynomials, which can therefore be assumed to have similar model quality. Furthermore, it can be seen that the maximal MSE value stays contained below 0.14, with the minimum R^2 value being above -0.06. Furthermore, it is of note that even though Ridge regression showed a degree-8 polynomial as optimal, which is similar to OLS, Ridge shows lower MSEs than OLS for appropriate values of λ. This is caused by the noise suppression properties of Ridge. As discussed in section II.A.3 this is made possible through Ridge's suppression of singular value modes. Modes with very low singular values compared to the penalty parameter λ are suppressed heavily, leading to models, which show less overfitting as compared to OLS.

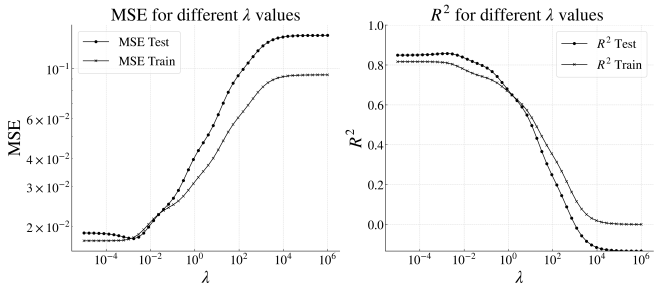


FIG. 5: Test-MSE (left) and Test-R^2 (right) for a polynomial of degree 8 fitted to the Runge function using Ridge regression with λ values ranging from 10^{-5} to 10^6.

The dependence of the MSE and the R^2 value on the parameter λ for the optimal polynomial degree of 8 can be seen in Figure 5. It can be seen that for large λ values, the training MSE is approaching a constant value, while the R^2 approaches 0 on the training set. This can be explained by the fact that as λ tends to infinity, all coefficients θ approach zero. Therefore, the model predicts a constant value of 0 for every training point (this value is zero as we centered our data, thereby dropping the intercept). The R^2 for the test data is slightly below zero, as we centered the model based on the mean of the training data, which slightly differs from the mean of the test data. Similar to before, the optimal value for the penalty parameter λ lies at approximately 0.018 (Note that we made no changes; this difference is caused by a less granular lambda grid). The plot furthermore nicely exhibits that a small dip in the MSE can arise, representing the optimal lambda values and a better performance than OLS.
However, it has to be noted that defining the optimal degree and the optimal value for the penalty parameter λ based on the lowest MSE on a single held out test split, as done for both the above OLS and Ridge models, leads to overfitting onto the test dataset. This can be solved using resampling techniques as explored in the following section.

C. The Bootstrap resampling technique and Cross-validation

1. Train and test MSE of different models

We computed the test and train MSE of the closed-form solution with the singular value decomposition to compare their behavior for different numbers of data points as well as different polynomial degrees. We used 80% of the data to train the model and 20% are testing.

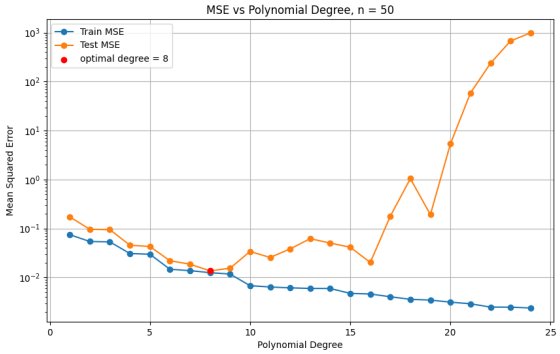


FIG. 6: Test and training MSE for $n = 50$

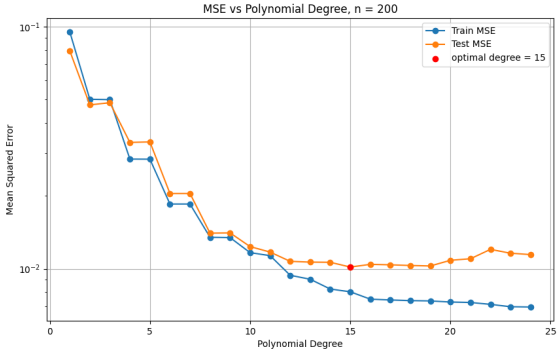


FIG. 7: Test and training MSE for $n = 200$

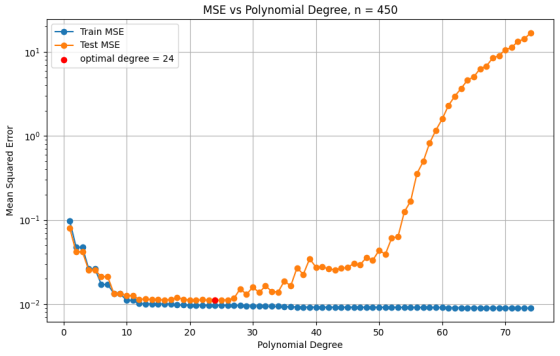


FIG. 8: Test and training MSE for $n = 450$

Figures 6, 7 and 8 show an important behavior: The training and the test MSE behave differently.

Train MSE: Here, we used the model to predict the exact same data points we used to train it. The model fits the exact data points including the noise, so the prediction gets better the higher the chosen polynomial degree (for huge degrees it may eventually become worse again due to numeric errors).

Test MSE: For the test MSE, we find a degree to achieve the best fit, when the test error is minimal. For higher degrees, the MSE becomes bigger again. This happens because the model again starts to fit the noise. However, as we tested it on data different from those we trained it on, it cannot predict the random noise on the never-before-seen data and therefore produces bigger errors again. This is called overfitting.

The plots show that the greater the amount of data points, the higher the polynomial degree can be without overfitting. This occurs because the model has more information about the position of the data points before it starts fitting the noise again. This behavior can be seen in the Figures. For 50 data points, the optimal polynomial degree is 8. For 200 data points, it is 15 and for 450 a degree of 24 gives the lowest test MSE.

2. The bias-variance decomposition

The mean MSE of the model, measured with models trained on different parts of the data using the bootstrap, can be decomposed into three contributing phenomena: bias, variance, and noise:

$$\mathbb{E}\left[(\boldsymbol{y} - \tilde{\boldsymbol{y}})^2\right] = \mathrm{Bias}^2[\tilde{y}] + \mathrm{var}[\tilde{y}] + \sigma^2,$$

with

$$\begin{aligned}\mathrm{Bias}^2[\tilde{y}] &= \mathbb{E}\left[\left(\boldsymbol{f} - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2\right], \\ \mathrm{var}[\tilde{y}] &= \mathbb{E}\left[\left(\tilde{\boldsymbol{y}} - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2\right] \\ &= \frac{1}{n}\sum_i \left(\tilde{y}_i - \mathbb{E}[\tilde{\boldsymbol{y}}]\right)^2.\end{aligned}$$

We calculated the mean MSE, bias and variance over many bootstraps and plotted them for different numbers of data points and polynomial degrees:

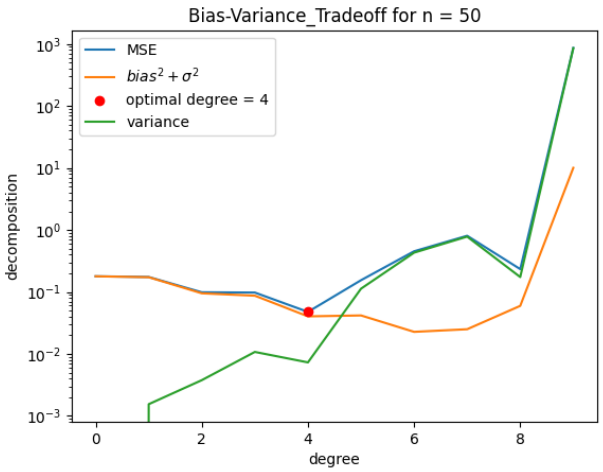


FIG. 9: Test and training MSE for $n = 50$

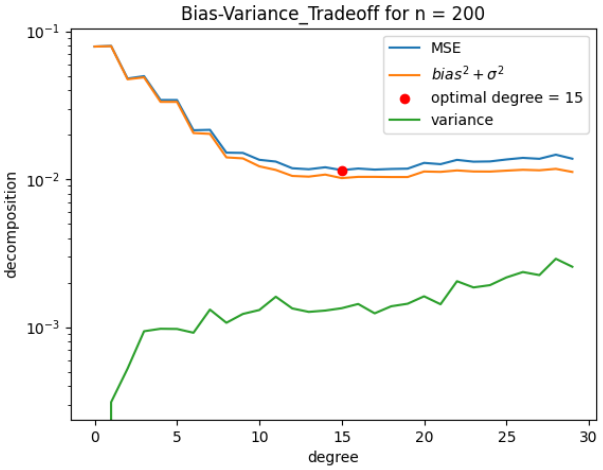


FIG. 10: Test and training MSE for $n = 200$

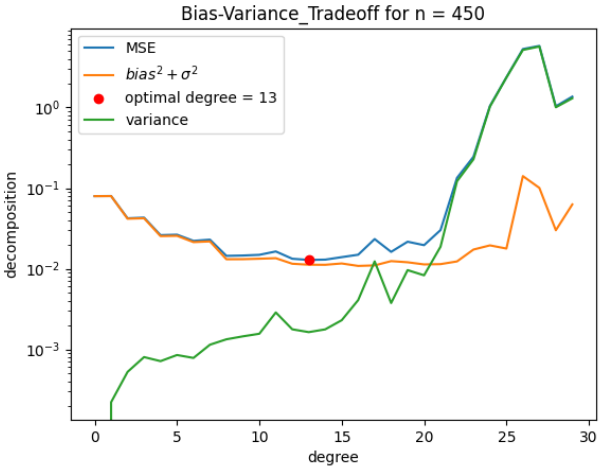


FIG. 11: Test and training MSE for $n = 450$

As proved above, the MSE contains three separate phenomena: bias, variance and the noise floor. In the following, we performed a bias-variance decomposition for the same data using the bootstrapping method (Figures 9, 10 and 11). In our case, the noise σ^2 is included in our mathematical description of the bias, which means that it actually becomes $bias^2 + \sigma^2$, because we cannot compare our result with the real value of the function due to the noise; we only compared our result with the test data. This gives the following optimal values for the degree: 4 for 50 points, 15 for 200 and 13 for 450. For 50 points, the optimal degree differs by a factor of 2 compared to before. However, the normal split of the data (a single 80/20 split) only produces 10 points to test the model on, so the optimal degree may differ a lot based on which data points end up in the test set and how noisy they are. For 200 data points, the optimal degree does not differ a lot compared to before, considering that the test MSE does not change much between degrees 13 to 20 for both methods. For 450 data points, the difference in the optimal degree is quite big, despite the larger amount of data points. For the normal train-test split, the test MSE again does not change much between degrees 12 and 26, so in terms of MSE, the difference between 13 and 24 would actually be small and could be caused by noise or how we split the data. The effect of the train-test split can be seen with the bootstrapping method. For the degrees above 20, the variance dominates the MSE. This means that different models predict way different values. This leads to a mean MSE over the bootstraps that consists of models whose predictions are fairly off as well as some that predict correctly even at higher degrees. If the model trained on the normal train-test split data is one of these well-predicting models for higher degrees, this does not conflict to the bootstrapping results. The bootstrap only gives means, and as the variance is high, the optimal degree may just differ a lot when only training it on one chosen train data set and not comparing with other ones. One other effect that can be seen is that the $bias^2 + \sigma^2$ starts to stay at a certain value for high degrees and the MSE is completely dominated by variance, whereas for low degrees, the $bias^2 + \sigma^2$ is responsible for most of the MSE. This has a good reason. For high degrees, the model fits noise that is normally distributed. The more noise it fits, the more the predictions of the model follow the randomization of the noise. This means the variance between the predictions of the model increases more and more the more heavily the model fits the noise. But the higher degrees can also pick up more information about the real data, so averaged over many models, the mean prediction still becomes better or stagnates, meaning the bias stays rather constant at some point. This can also be seen in the Figures. Theoretically, the bias may converge to 0 for higher and higher degrees if the generating function can be fully described by polynomial features. For low degrees, the complexity of the model is fairly low, so choosing another set of training data hardly affects the prediction, as the difference in behavior of the data is small compared to what the model can depict, so it needs higher model complexity to describe it. This results in the variance between models being fairly low for different bootstraps and the MSE mainly consists of bias. This opposing behavior of bias and variance also means that the best model for some problems is not necessarily the one with the lowest test MSE. One may be willing to trade a slightly higher MSE for less variance or bias. But, of course, the MSE is what we optimize for in the first place, so this hardly depends on what the model is made for.

We performed a cross validation for OLS and Ridge regression to find the MSE, the optimal λ and the correct polynomial degree of the models.

3. Cross-validation for OLS regression

The following Figures show the OLS MSE as a function of the polynomial degree, cross-validated over 5 and 10 folds:

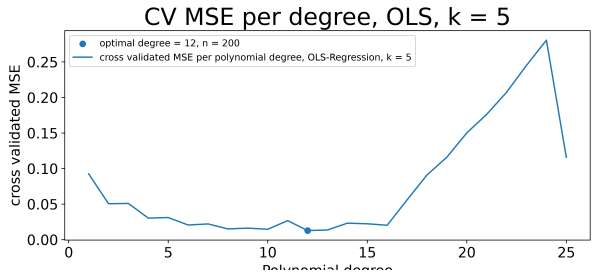


FIG. 12: Cross-validated MSE with 5 folds for OLS regression

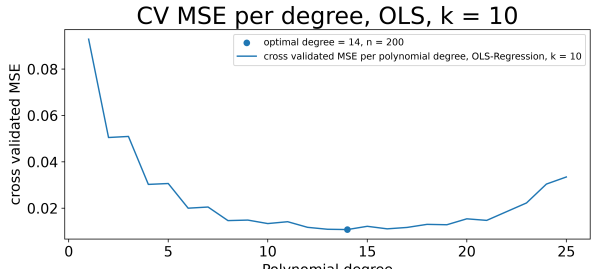


FIG. 13: Cross-validated MSE with 10 folds for OLS regression

The plots in Figures 12 and 13 point towards similar optimal degrees, which are also similar to the degree we obtained using the bootstrap. For 5 folds, the lowest MSE is given at degree 12, for 10 folds at degree 14 and for the bootstrap at degree 15. As the MSE does not significantly change between degrees 12 and 15, these three results are very reasonable and point towards a similar degree of model complexity for n = 200 data points. The results lie relatively close to each other as the MSE gets evaluated for models trained on different datasets and then averaged over the sets for both methods. In order to scale the data for the cross-validation, we used the scikit learn functions StandardScaler() for x and TransformedTargetRegressor() for y inside the pipeline functionality of scikit learn (Pedregosa *et al.*, 2011). This is very important because the data have to be scaled separately for every run of the cross validation, and both the training and the test sets have to be scaled with the mean and standard deviation of the training set. Otherwise, the mean and standard deviation used to scale the data would be calculated on data the model is not meant so see until the test run, which would lead to data leakage and therefore wrong results.

4. Cross-validation for Ridge regression

As the optimal degree for our model seems to be between 12 to 15, we determined the optimal λ for Ridge regression with models of these degrees. However, we did not choose an optimal value of λ for the lowest MSE, but marked all possible values of λ that produce an MSE which is at most one standard deviation away from the minimal value (Hjorth-Jensen, 2026):

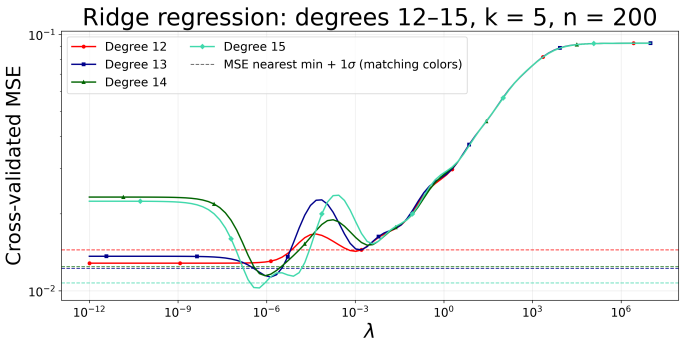


FIG. 14: Cross-validated λ with 5 folds for degrees 12 to 15 for Ridge regression

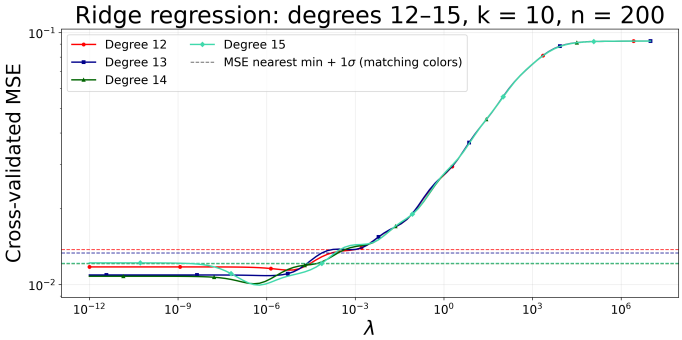


FIG. 15: Cross-validated λ with 10 folds for degrees 12 to 15 for Ridge regression

Figures 14 and 15 show the optimal values of λ that lie at or under the respective lines. The curves for k = 5 and k = 10 are quite different from each other. There are two effects that come with choosing the optimal value for k. A higher value of k results in more individual test sets, which are used to determine the mean MSE of the model, so fluctuations between the models are averaged out better. However, a higher k also means that fewer points end up in the test set; for k = n the test set would only include one single point. This drives the variance between test MSEs of the models up as the differences between the test sets become bigger the smaller their size. This leads to different results for the optimal λ for different values of k. In Figures 14 and 15 the curve smoothens out for the higher k value. This could be due to the averaging over more folds, as Figure 15 averages

over twice as many folds as Figure 14. For k = 5, the optimal λ values lie between 0 and about $10^{-2.5}$, while for k = 10, they lie between 0 and 10^{-3}. This means that there are models that predict better with OLS rather than Ridge regression with $\lambda > 0$. This behavior can be seen for degrees 12 in Figure 14 and 13 in Figure 15. For the other degrees, a λ significantly larger than 0 (meaning larger than errors due to machine precision) is optimal. This fits the observation that the lowest MSEs lie around 10^{-2} for both Ridge and OLS regression. The lowest error occurs at k = 10 for the degree-15 polynomial at 10^{-2} with a λ of about 10^{-6}. The exact values for these are discussed in section III.G.

D. Convergence Dynamics and Hyperparameter Sensitivity of Modern First-Order Optimizers

Comparing the analytical and the AD gradients of the OLS and Ridge cost functions for the degree-5 polynomial fit of the Runge function yields the maximal deviations listed in table I.

	$\max_j \lvert g_{\mathrm{AD},j} - g_{\mathrm{analytic},j} \rvert$
OLS	5.6×10^{-17}
Ridge	1.11×10^{-16}

TABLE I: Maximal absolute deviation between the gradients obtained by automatic differentiation and analytically.

This agrees with our expectation that automatic differentiation yields the gradient up to machine precision, i.e. with deviations of $\mathcal{O}(10^{-16})$.

1. Comparison of different gradient descent methods

	λ_{min}	λ_{max}	γ_{max}	γ^*	κ
OLS	0.0109	5.6527	0.3538	0.3531	520.34
Ridge	0.2109	5.8527	0.3417	0.3298	27.76

TABLE II: Eigenvalues of the Hessian and the largest safe learning rate for plain gradient descent.

For the Runge function with additive normally distributed noise (standard deviation 0.1) and the OLS and Ridge cost functions for a polynomial of degree 5, the smallest and largest eigenvalues of each Hessian are given in table II. The table also lists the maximum and optimal learning rates, calculated from eqs. (18) to (19).

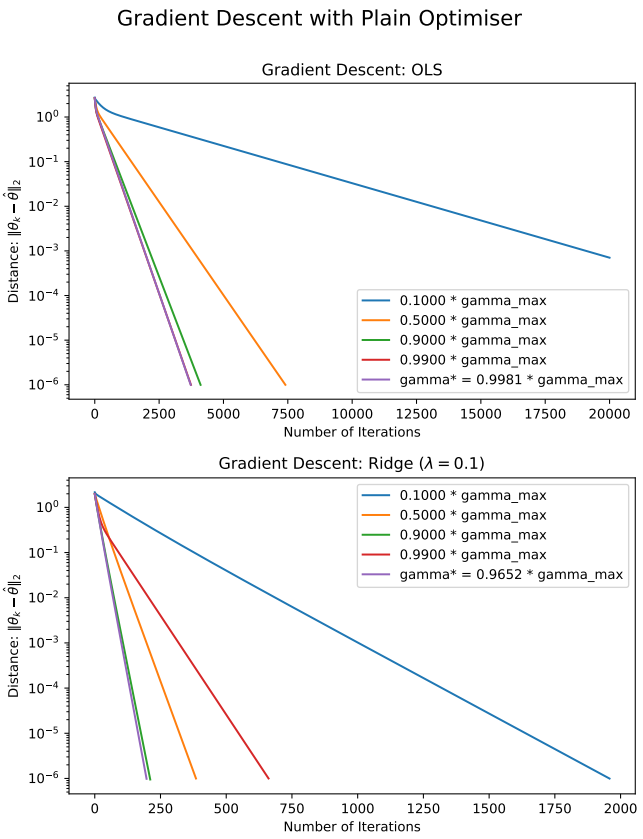


FIG. 16: Distance of plain GD to the closed-form solution of OLS (top) and Ridge (bottom) versus the number of iterations for different learning rates. $\gamma_{\max}$ and γ^* are taken from table II.

a. Plain GD Figure 16 shows the convergence of plain gradient descent towards the OLS and Ridge solutions for different learning rates. Since all learning rates are chosen below $\gamma_{\max}$, none of the runs diverge. However, we observe a first sign that plain gradient descent becomes slow for small learning rates: for OLS, $\gamma = 0.1 \cdot \gamma_{\max}$ fails to converge within 20 000 iterations. Additionally, convergence is faster for Ridge than for OLS and less sensitive to the learning rate, which we expected from the smaller condition number argument.

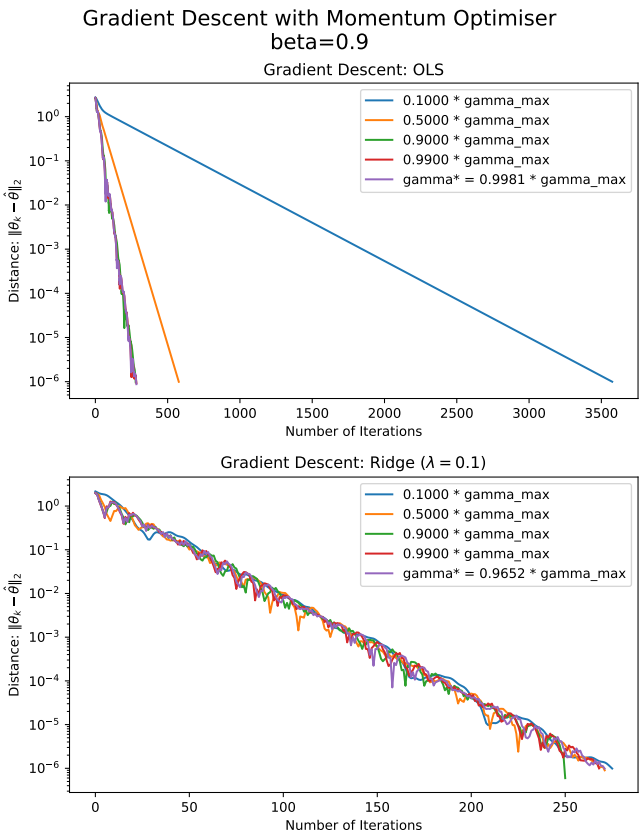


FIG. 17: Distance of momentum GD to the closed-form solution of OLS (top) and Ridge (bottom) versus the number of iterations for different learning rates. $\gamma_{\max}$ and γ^* are the maximum and optimal learning rates of plain gradient descent.

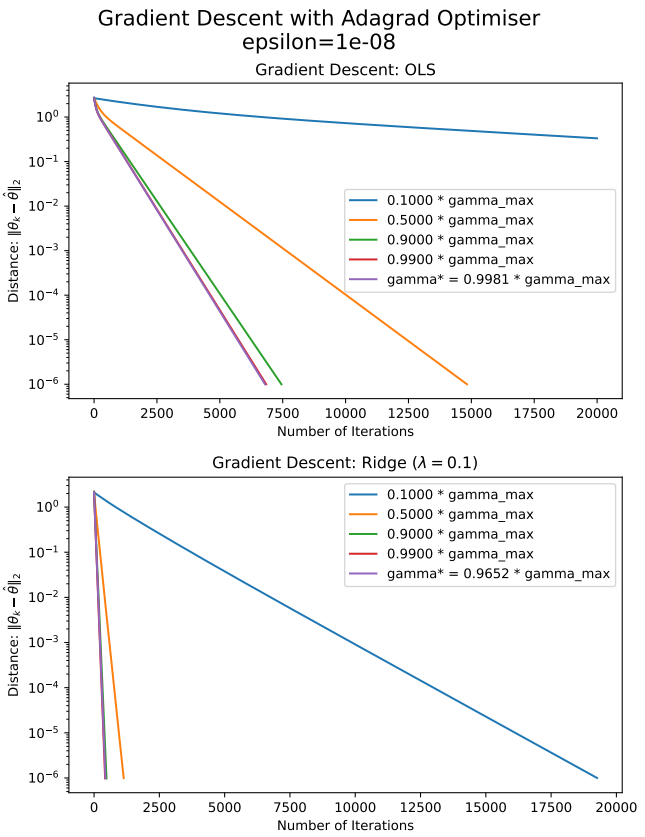


FIG. 18: Distance of AdaGrad GD to the closed-form solution of OLS (top) and Ridge (bottom) versus the number of iterations for different learning rates. $\gamma_{\max}$ and γ^* are the maximum and optimal learning rates of plain gradient descent.

b. Momentum GD As expected, adding a momentum term increases the convergence speed for almost all learning rates. For OLS, it reduces the number of required iterations by an order of magnitude and also leads to convergence for $\gamma = 0.1 \cdot \gamma_{\max}$. Ridge regression is also accelerated by momentum, with one exception: at $\gamma = \gamma^*$, plain gradient descent converges in 198 iterations, whereas momentum requires 272 (see tables VII to VIII in section C). This is expected and exemplifies that the optimal learning rate of plain gradient descent is no longer optimal for other gradient descent methods.

c. AdaGrad GD We expected AdaGrad to be prone to getting stuck due to its infinite memory. For the quadratic problems considered here, however, this effect is of minor importance: only OLS with $\gamma = 0.1 \cdot \gamma_{\max}$ slows down early on and does not even reach $\left\|\boldsymbol{\theta}_t - \hat{\boldsymbol{\theta}}\right\| \leq 10^{-1}$ within 20 000 iterations.

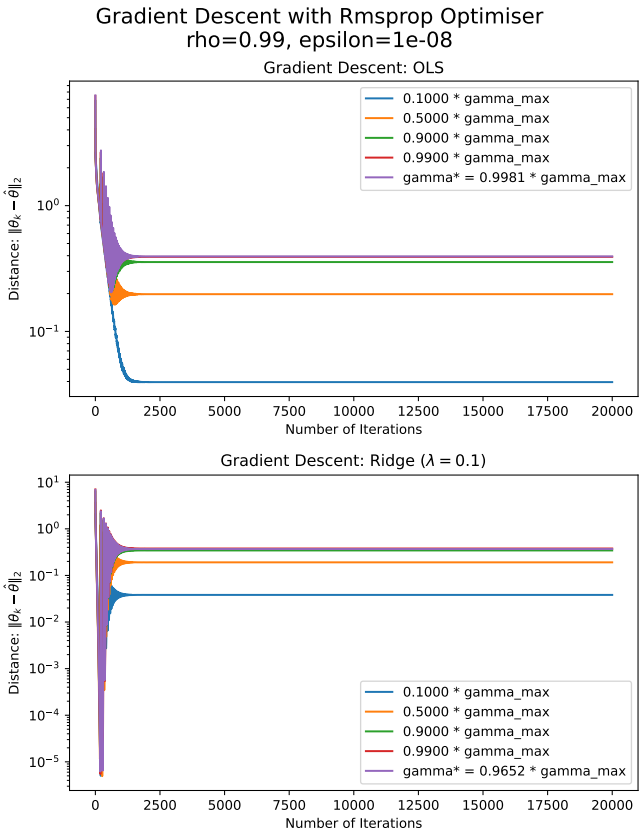


FIG. 19: Distance of RMSprop GD to the closed-form solution of OLS (top) and Ridge (bottom) versus the number of iterations for different learning rates. $\gamma_{\max}$ and γ^* are the maximum and optimal learning rates of plain gradient descent.

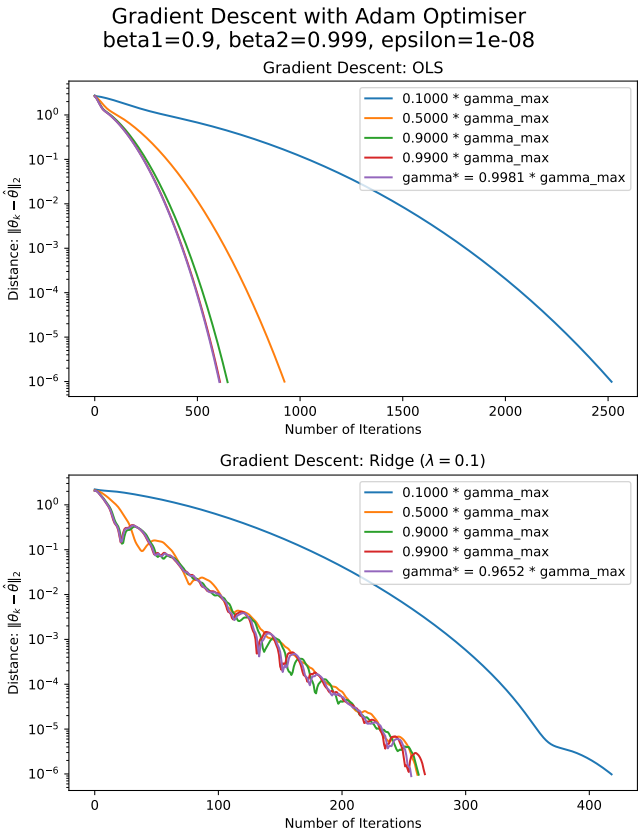


FIG. 20: Distance of Adam GD to the closed-form solution of OLS (top) and Ridge (bottom) versus the number of iterations for different learning rates. $\gamma_{\max}$ and γ^* are the maximum and optimal learning rates of plain gradient descent.

d. RMSprop GD RMSprop "forgets", i.e. it has no infinite memory, but in turn loses the ability to settle. This expectation is reflected in fig. 19: the distance oscillates in the beginning and reaches its minimum early on, with the oscillations being particularly dominant for Ridge regression. Afterwards, the distance levels off at a value of the order of the learning rate, as expected. Consequently, none of the learning rates leads to a distance below the convergence tolerance.

e. Adam GD Adam converges for all learning rates considered. Its convergence speed is of the same order as that of momentum. However, Adam's real strength only becomes apparent when comparing all methods over a wider range of learning rates.

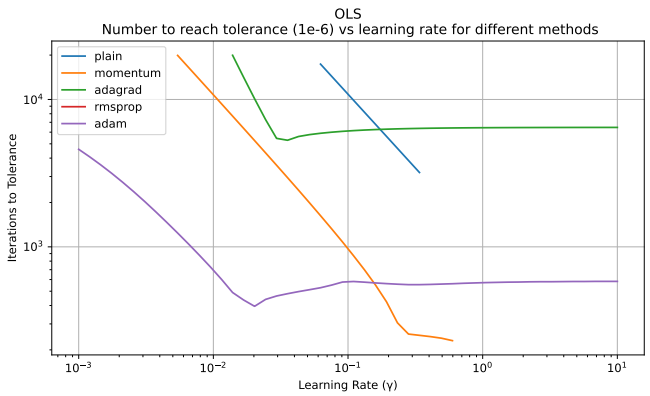


FIG. 21: Number of iterations required by each method to reach $\left\|\boldsymbol{\theta}_t - \hat{\boldsymbol{\theta}}\right\| \leq 10^{-6}$ for OLS regression, as a function of the learning rate. Runs that do not meet the convergence criterion within 20 000 iterations are omitted. RMSprop does not converge for any learning rate and therefore does not appear in the plot.

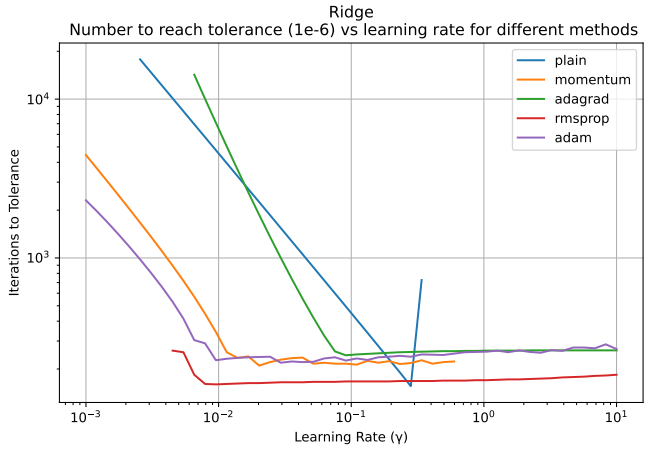


FIG. 22: Number of iterations required by each method to reach $\left\|\boldsymbol{\theta}_t - \hat{\boldsymbol{\theta}}\right\| \leq 10^{-6}$ for Ridge regression, as a function of the learning rate. Runs that do not meet the convergence criterion within 20 000 iterations are omitted.

f. Comparison of all GD methods and learning rates Comparing the number of iterations until convergence for all five methods over learning rates $\gamma \in [10^{-3}, 10^{1}]$ for OLS and Ridge allows a more detailed assessment of which method is advantageous in which regime. Focusing on OLS first: as seen before, RMSprop does not settle close enough to the minimum to meet the convergence criterion within 20 000 iterations and therefore does not appear in fig. 21 at all. For Ridge, however, RMSprop shows its merits in certain cases. The initial oscillation of the distance starts off with a steeply falling slope, and empirically, for Ridge, this initial descent even reaches distances below 10^{-7}. As fig. 22 shows, RMSprop requires the fewest iterations of all methods for learning rates roughly in $[10^{-2}, 10^{1}]$, with the exception of plain gradient descent at the optimal learning rate γ^*. However, this happens during the initial oscillations: the distance drops steeply, before RMSprop leaves the tolerance region again and levels off at a distance of the order of γ. The corresponding iteration counts in fig. 22 therefore mark a transient crossing rather than convergence, which in practice could not be detected without knowing $\hat{\boldsymbol{\theta}}$. RMSprop is thus useful when a coarse tolerance of the order of γ suffices, as it approaches the minimum quickly, but it does not converge to high accuracy.

As expected, Adam is the least sensitive to the learning rate of all methods examined: for both OLS and Ridge, it converges for all values of γ. For learning rates below roughly 10^{-1} (OLS) and 10^{-2} (Ridge), Adam also requires the fewest iterations.

In contrast, plain gradient descent is the most sensitive in both cases, as expected. Within 20 000 iterations, it only converges for $\gamma \in [5 \times 10^{-2}, \gamma_{\max}^{\mathrm{OLS}} \approx 0.3538]$ for OLS and $\gamma \in [2 \times 10^{-3}, \gamma_{\max}^{\mathrm{Ridge}} \approx 0.3417]$ for Ridge. However, for Ridge at the optimal learning rate γ^* (see table II), plain gradient descent converges faster than all other methods.

Apart from Ridge at the optimal learning rate, momentum is always faster than plain gradient descent, as expected.

AdaGrad also shows little sensitivity to the learning rate. Although its infinite memory makes it more prone to getting stuck, it converges as long as $\gamma \gtrsim 10^{-2}$. In particular, for $\gamma \gtrsim 10^{-1}$, the accumulated squared gradients dominate the convergence speed, which then becomes almost independent of the learning rate.

E. Lasso Regression and discussion of different gradient descent approaches

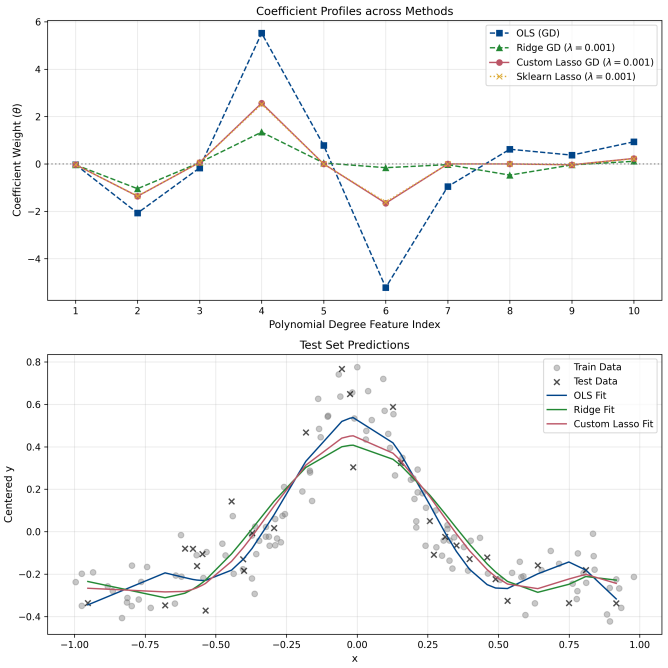


FIG. 23: Figure of the coefficient weights for each approach and of the prediction for unseen data

Figure 23 shows both the coefficient weights and the prediction for unseen data for each approach. The focus is placed on the comparison of our own Lasso regression algorithm with the implementation of Lasso regression in *scikit-learn*. As discussed in section II, Lasso has the capacity to set features to exactly zero, whereas Ridge only drives them towards zero.

1. Discussion of OLS, *scikit-learn*-Lasso, Ridge and the self-written Implementation

Starting with the OLS fit, which can be seen in the diagram of Figure 23, we find that the coefficients are spiking up to 5 and -5 to cancel each other, as the model is overfitting. It tries to fit every minor detail rather than smoothing out the function. This happens because the features x, x^3, x^5, x^7,... are highly correlated. With an increasing polynomial degree, X^TX gets ill-conditioned; even small changes in noise cause large jumps in the calculated weights. Additionally, the product of those matrices becomes nearly singular, which makes matrix inversion an unstable operation and the number between largest and smallest eigenvalues becomes very large. This can lead to a loss of numerical precision as the smallest eigenvalue of the to be inverted matrix drops close to zero.

Ridge adds a squared penalty term to the cost function, shrinking all coefficients towards zero. By doing so, the matrix is not ill-conditioned anymore, as adding the penalty term guarantees the invertibility even when X^TX is singular. Furthermore, all eigenvalues are shifted by 2λ, directly capping the condition number and suppressing numerical instability. This prevents the oscillations of the coefficients and effectively suppresses the boundary "wiggles", visible in the right and left diagram of Figure 23. In the right diagram, the oscillations near the boundaries are most prominent in the blue line belonging to the OLS fit. Through this, the regression trades a bit of bias for a large decrease in the variance.

Lasso is similar to Ridge; the difference between the two is the penalty term, which is not added as a squared sum of the coefficients rather than as the sum of the absolute values. This method can shrink coefficients to zero more effectively, which makes it extremely powerful when fitting data with a lot of unnecessary features. In the left diagram of Figure 23, this behavior becomes obvious. The yellow and red lines belonging to the Lasso regression shrink almost half of all features to zero.
Lasso does not have a closed-form solution such as Ridge; therefore an iterative algorithm is required (here, we used the sub-gradient descent that starts at an initial guess and updates every step until convergence is reached). Depending on the problem, iterative algorithms often need more time as a single matrix inversion, which can make the regression speed much slower. Furthermore, they require parameters like the learning rate, the maximum number of iterations and a tolerance threshold. These algorithms stop when they reach a tolerance threshold, yielding a numerical approximation rather than an exact result. The non-differentiability of the gradient of the Lasso cost function has to be solved at $\theta_j = 0$. Because the function $|\theta|$ forms a sharp kink at 0, the standard derivative $\frac{d}{d\theta}|\theta|$ does not exist at that exact point. We handled this problem by introducing a sub-gradient.

$$\boldsymbol{g} = \frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}) + \lambda \text{sign}(\boldsymbol{\theta})$$

Instead of using a single derivative, we decided to use a sub-gradient, defined as:

$$g \in \begin{cases} \{1\} & \text{if } \theta > 0 \\ \{-1\} & \text{if } \theta < 0 \\ [-1, 1] & \text{if } \theta = 0 \end{cases}$$

This was implemented by choosing $g = 0$ and $\theta = 0$, which picks a sub-gradient element that is an element of the subdifferential set $\partial|\theta|$ at the origin as *np.sign(0) = 0* implicitly selects $0 \in [-1, 1]$. However, this method tends to bounce back and forth across zero, causing it to converge slowly and struggle with setting parameters to exactly 0. The code uses Adam's adaptive moment estimates (m_t, r_t) to dynamically smooth the learning rates

per parameter, enabling stable convergence near sharp kinks.

2. Comparison of the Lasso code with *scikit-learn*

TABLE III: Rounded values of each feature for every regression method.

Degree	Custom Lasso	Sklearn	OLS	Ridge
x^1	-0.030	-0.031	-0.026	-0.035
x^2	-1.363	-1.353	-2.073	-1.046
x^3	0.061	0.061	-0.166	0.057
x^4	2.567	2.523	5.520	1.344
x^5	0.001	0.000	0.782	0.040
x^6	-1.657	-1.609	-5.215	-0.158
x^7	0.000	0.000	-0.955	-0.030
x^8	-0.001	0.000	0.620	-0.472
x^9	-0.036	-0.036	0.372	-0.041
x^{10}	0.232	0.217	0.935	0.111

Our implementation as well as the jax (Bradbury *et al.*, 2018) automatic differentiation (discussed later in section III.E.3) are using sub-gradient descent combined with Adam, it updates all coefficient parameters simultaneously in each step using adaptive learning rates and first-order gradient approximations. This method can struggle at non-differentiable points and momentum in Adam causes parameters to bounce around zero. Thus, coefficients are rarely driven exactly to zero but to extremely small values (This can be observed in table III, our own Lasso code was not able to set any feature to exactly 0.0, whereas the *scikit-learn* code was able to.). In comparison, *scikit-learn* uses coordinate descent written in C that optimizes one feature at a time while holding all other features fixed, solving each single-variable subproblem analytically. As it uses the analytical soft-thresholding operator ($\mathcal{S}_\gamma(z) = \text{sign}(z)\max(|z| - \gamma, 0)$), this method can set features to exactly zero. (Pedregosa *et al.*, 2011) However, this makes no large difference for the fits of the Runge function (see table IV and Figure 23).

3. Confirming the results with JAX Automatic Differentiation Verification

First of all, to evaluate the quality of the fit the data was split in Test and Train Data, however instead of relying on a single random 80/20 split, the code uses k-fold cross validation to evaluate the model across five distinct data partitions. The values in Table IV represent the average MSE across all five iterations to eliminate single split noise. The model evaluates the fits, as mentioned, with cross validation MSE, CV Test MSE measures how precisely the model predicts unseen holdout data. Furthermore, the model stability is measured with standard deviation (std). A small standard deviation indicates a robust, consistent and not overly sensitive model. Evaluating *jax.grad(jnp.abs)(0.0)* returns *(1.0)*,

TABLE IV: 5-Fold CV MSE performance across regression methods ($\lambda = 0.001$, $d = 10$). Standard deviations are in parentheses.

Method	**Train MSE**	**Test MSE**
OLS (JAX AD)	0.012992 (0.0007)	0.014579 (0.0029)
Ridge (JAX AD)	0.020211 (0.0010)	0.022620 (0.0055)
Lasso (JAX AD)	0.017146 (0.0008)	0.019127 (0.0045)
Lasso (Scikit-Learn)	0.017220 (0.0008)	0.019182 (0.0045)

which confirms that professional AD engines handle non-differentiable points by assigning default sub-gradient choices at singular coordinates. Matching JAX auto-diff gradients against manual vector implementations verifies that the custom loss and sub-gradient functions contain zero algebraic or code errors (which can be seen in table IV). In conclusion, when using JAX's AD gradients to minimize the Lasso loss function, it achieves the exact same cross-validation test MSE (0.019127) as the manual sub-gradient code. This confirms that treating $g = 0$ at $\theta_j = 0$ successfully drives the optimizer to the global minimum.

F. Stochastic gradient descent in comparison with closed forms and full gradient descent

1. Comparison of different mini-batch sizes

To compare different mini-batch sizes, we used the Runge function with $N = 100$ and added noise. The data was split into test and training sets (80% training and 20% test) to run different algorithms. So a full gradient would run with $N = 80$

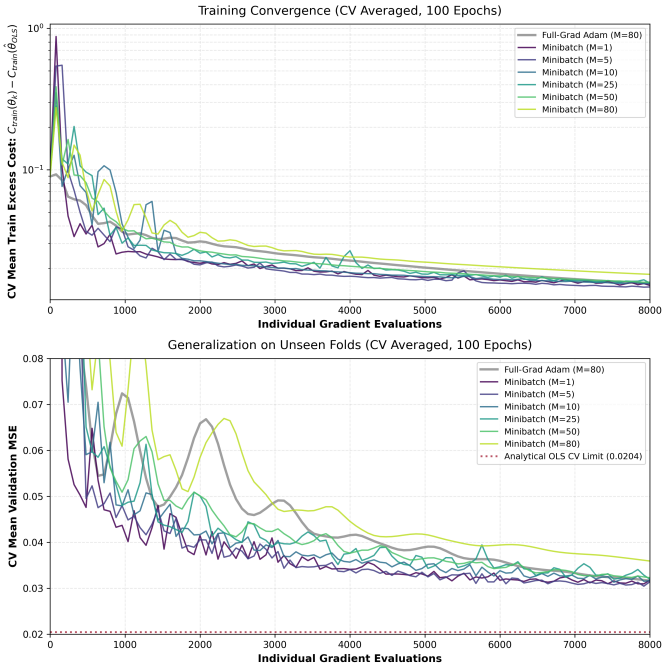


FIG. 24: Excess cost for $n_{epochs} = 100$ and the calculated MSE for the test data, compared with full gradient descent and closed-form OLS

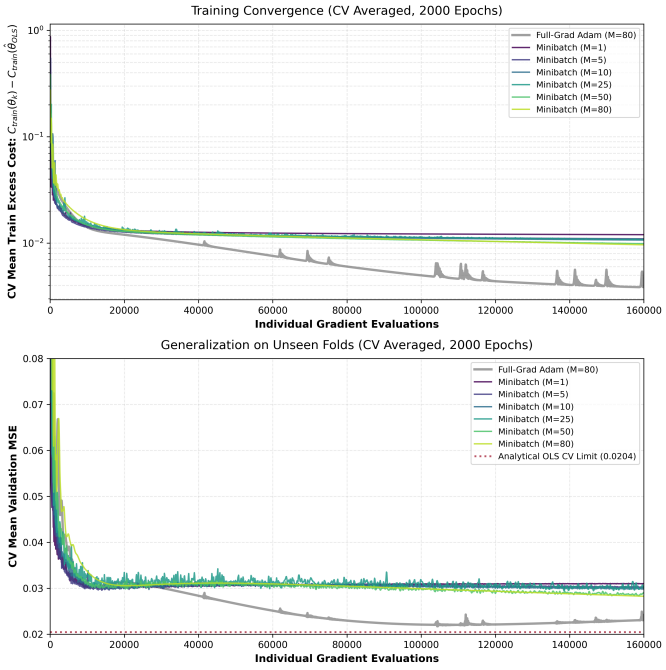


FIG. 25: Excess cost for $n_{epochs} = 2000$ and the calculated MSE for the test data, compared with full gradient descent and closed-form OLS

Looking at the Figures 24 and 25, we see that two things become obvious. Firstly, for smaller mini-batches, the parameters are updated faster, whereas for bigger batches or even the full gradient descent, the algorithm has to calculate a lot before updating any parameter. Therefore, the excess cost is shrinking faster for smaller batches. Secondly, only the excess cost for full gradient descent is decreasing, even after 80000 evaluations. Every mini-batch converges at similar excess cost of around $\approx 10^{-2}$. As every mini-batch $M < 80$ only uses limited data points, the gradient is bouncing around the minimum, but never able to reach it, because this introduces random variance at every single step. The figure also compares to Full Gradient Adam, but even with Adam's adaptive learning rate, this variance prevents the parameter from settling at the exact analytical minimum θ_{OLS}. However, the full gradient descent is using all available data points, setting the variance to exact zero. Adam can continuously redefine the learning rate so the full gradient will converge towards the analytical solution. Full-batch Adam can produce spikes when its updates overshoot a low-cost region, particularly if the learning rate is large or momentum carries the parameters too far. The logarithmic y-axis of the upper plot of Figure 24 makes these changes look more pronounced because it emphasizes relative differences, especially when the excess cost is already small. Minibatch methods tend to show more frequent fluctuations because each update uses only part of the data. Different batches give different gradient estimates, so the parameter updates can vary from step to step. At an epoch boundary, the data are reshuffled, and the next batch may produce a noticeably different update.

2. Evaluating the fit quality

To evaluate the quality of the fit, both Figures (24 and 25) have a second plot that shows the mean cross-validation MSE. Plotting the error on the test data across the k folds helps to visualize the true and robust generalization error trajectory. Thereby preventing, e.g., overfitting at any epoch because the curve would be bending upwards after dipping down, as Figure 25 displays, when the gray curve (Full Gradient Descent with Adam) is rising again after $\approx 110\,000$ evaluations. This is a demonstration of overfitting as the curve hits a minimum at ≈ 0.0220 and then starts rising again towards ≈ 0.0230. This happens because the model fits the noise instead of the signal. Early in training, the model learns the main underlying Runge function; however past ≈ 110.000 evaluations, the unregularized model starts fine-tuning its weights to fit small random noise, deteriorating its performance on unseen validation folds. We can prevent this from happening, by either stopping after $\approx 100\,000$ evaluations, penalizing large weights to prevent coefficients from exploding to fit noise or by decaying γ late in training to stop Adam from taking large steps in steep regions.

3. Dependence on the learning rate schedule

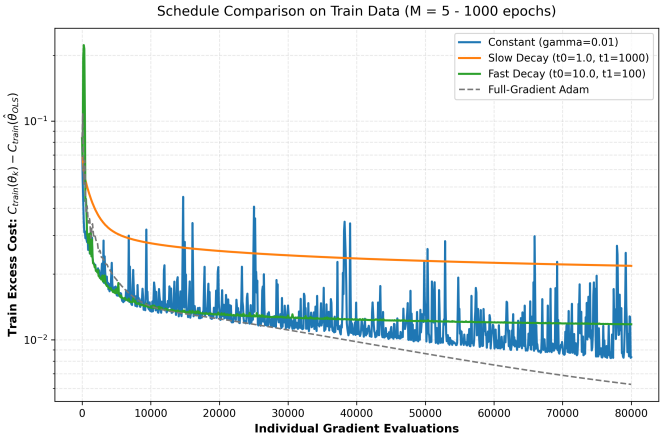


FIG. 26: Excess training costs for different learning rate schedules for each evaluation step (80 000 evaluations in total). We used a batch size of $M = 5$

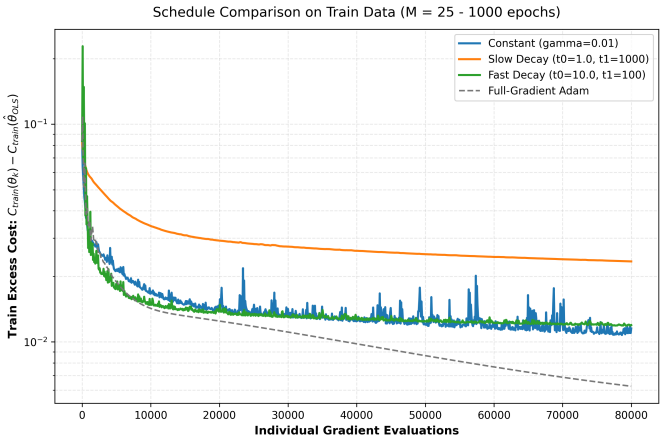


FIG. 27: Excess training costs for different learning rate schedules for each evaluation step (80 000 evaluations in total). We used a batch size of $M = 25$

To compare different learning rate schedules, Figure 26 and Figure 27 display the excess costs for the training data for different batch sizes and different learning rate schedules for each evaluation.

For Figure 26, we used a batch size of five. For a constant learning rate schedule, this results in volatility and frequent noise spikes due to the variance of small batches. Increasing the batch size to $M = 25$ (as shown in Figure 27) prevents those oscillations and provides sufficient gradient averaging, yielding a much smoother trajectory.

The fast decay ($t_0 = 10.0, t_1 = 100$) appears to be the fastest across both batches. It preserves large step sizes early for rapid initial convergence and then decays quickly enough to stabilize the weights and closely track the full-grad Adam benchmarks.

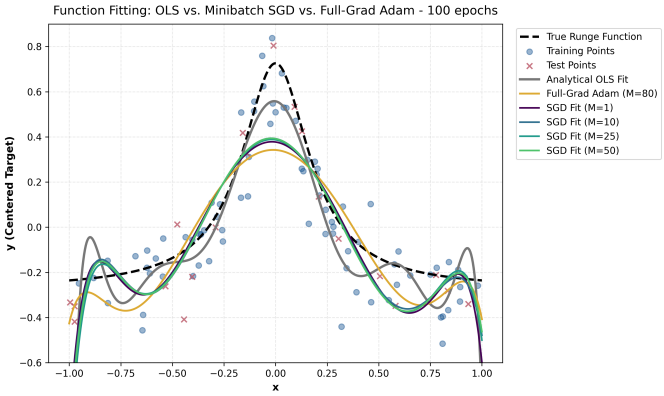


FIG. 28: Visual conclusion of the different methods for 1000 epochs including training as well as test data points.

The slow decay ($t_0 = 1.0, t_1 = 1000$) severely stalls high above the other schedules in both configurations. Because the initial step size is too small, the parameter updates remain too conservative to converge within 80 000 evaluations.

When comparing different batch sizes, the small batch computes five times more parameter steps per epoch than the big batch. Thus, smaller batch sizes decay faster per epoch, highlighting why schedule hyperparameters (t_0, t_1) must be recalibrated whenever batch size is altered.

4. Final accuracy relative to closed-form-solutions

To demonstrate the behavior of stochastic and full-batch optimization, the cross-validated training excess cost and validation MSE were tracked across training durations from 100 to 2000 epochs. While Full-Gradient Adam continuously minimizes training excess cost—dropping from 1.56×10^{-2} at 100 epochs down to 3.86×10^{-3} at 2000 epochs—a critical distinction must be drawn regarding generalization error. Blindly extending training introduces severe overfitting. As shown by cross-validation in figure 25, Full-Gradient Adam reaches its optimal generalization minimum at $\approx 110,000$ gradient evaluations (MSE ≈ 0.0220). Beyond this threshold, the unregularized model begins fine-tuning its parameters to memorize random noise rather than the underlying Runge signal, causing the test error to bend upward to 0.0231 by 2000 epochs. In contrast, mini-batch SGD ($M = 25$) plateaus near a train excess cost of 1.08×10^{-2} and a validation MSE of 0.0300, illustrating how mini-batch variance prevents SGD from settling near either the early-stopping minimum of Full-Grad Adam or the exact analytical OLS baseline (MSE = 0.0204). The ex-

plicit data can be found in table XII in the appendix.

As discussed in section III.A, OLS evaluates the exact global minimum of the unregularized MSE, whereas Ridge regression (section III.B) modifies the optimal baseline by applying a penalty parameter λ, shrinking the singular-value modes to suppress the high-variance oscillations of a high-degree polynomial. The underlying algorithm depends on a matrix inversion. This operation scales cubically with the number of features; solving this for e.g. $p = 11$ requires only small computational overhead. Thus, for a low-dimensional task like this, calculating the exact analytical solution is much faster and strictly more accurate than running thousands of numerical optimization loops.

Full-gradient Adam, configured with a constant step size, successfully converges towards the theoretical OLS minimum. But as it lacks the explicit λ penalty, it overfits by memorizing the normally distributed noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$ rather than the underlying Runge function — though even without noise, high-degree polynomial OLS still collapses on the Runge function. This results in degraded test accuracy if early stopping is not applied.

The mini-batch SGD never reaches the OLS accuracy limits. The variance of single-sample or small batch gradient estimates, combined with a decaying learning rate schedule, blocked the optimizer's progress. This results in bouncing around the minimum, never reaching the precise mathematical minimum guaranteed with OLS.

Iterative methods like SGD compute updates by using a matrix-vector product which scales linearly with the batch size M and feature count p. This linear scaling becomes necessary when transitioning to larger numerical algorithms or data sets. In large systems, tracking the exact Hessian matrix for direct eigensolvers exhausts memory and is computationally infeasible. The $\mathcal{O}(Mp)$ cost of mini-batch gradients circumvents the $\mathcal{O}(p^3)$ bottleneck entirely.

5. Limitations and Proposed Enhancements

As we did not use different shuffled seeds and build the average of them, the final MSE represents a single sample path rather than the expected behavior of the algorithm. When only comparing on a single seed, there is a risk of overfitting the hyperparameters to that specific state and data order. A superior looking schedule might be less robust on average than a competitor. Furthermore, to perfectly compare SGD to a closed deterministic form, we should quantify the variance of the final accuracy. Relying on a single seed removes the ability to discuss how heavily the SGD accuracy fluctuates.

To compare the results visually, Figure 28 compares the fit quality for the Runge function for closed and iterative forms. This Figure displays the main conclusion of this task pretty well: obviously, OLS is overfitting the function. This becomes visible at the boundaries of the problem, as the OLS function strongly oscillates.

A Lasso regression model, solved using stochastic gradient descent or full gradient Adam can partly solve this problem by reducing the oscillation, but still is not managing to fully prevent this.

Unregularized full-gradient Adam fails here by eventually overfitting and memorizing normally distributed noise, while standard mini-batch SGD stalls and bounces around the minimum due to the high variance of small batch gradient estimates. To make the optimization process more robust and stable, modern methods apply strict validation-based early stopping to halt training before noise memorization begins, and utilize decoupled weight decay (such as in AdamW) to introduce an explicit penalty without interfering with adaptive learning rates.

G. Final comparison between OLS, Ridge and Lasso Regression with cross validation

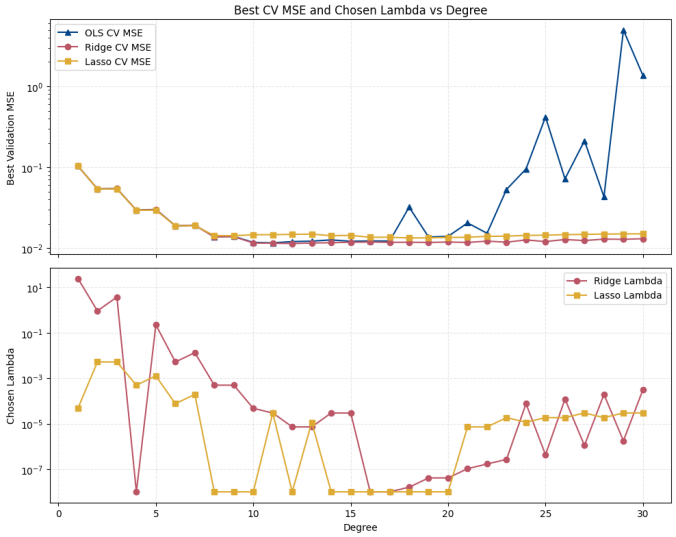


FIG. 29: Lowest CV-MSE for OLS, Ridge and Lasso with optimal values for the penalty parameter λ plotted against the degree of polynomial using polynomial degrees from 1 to 30.

Figure 29 displays the best test MSE for each degree. For Lasso and Ridge regression, the lower part of the plot also plots the optimal lambda. We produced this plot using with k-fold cross-validation and Lasso and Ridge regression from *scikit-learn*, even though we implemented a Lasso Regression above in section III.E. This has a simple reason: *scikit-learn* is natively written and compiled in *C/Cython*, so it compiles directly to native machine code executed by the CPU. Additionally, its Lasso uses advanced algorithmic optimizations like active-set coor-

dinate descent and zero-copy memory views that dramatically reduce computation and memory overhead. The used code uses a loop over every λ in a range of 10^{-8} - 10^{2} with 50 grid points in total to find the best λ for each degree. This λ was then used for the plot in Figure 29. The plot can be divided into three areas:

- degree 0 - $\approx$ 8: underfitting
- degree 8 - $\approx$ 17: good fit
- degree $>$ 17: overfitting

The first area has a high MSE starting at 10^{-1} and is decreasing almost linearly to $\approx 10^{-2}$. All fits (OLS, Ridge and Lasso Regression) are almost identical. For Ridge and Lasso, the penalty terms are also decreasing but are still higher in comparison to higher-degree polynomials. Because the fit can't catch every detail from the data, the penalty term has to be small so it won't suppress even more details. Except for λ at a degree of four, Ridge always has a higher penalty term than Lasso. The L_2 penalty shrinks coefficients smoothly and proportionally. Even with a larger λ (e.g., 10^{1} or 10^{0}), Ridge keeps all terms active with small weights. The L_1 penalty forces coefficients exactly to zero (at least when using the *scikit-learn* implementation). If Lasso chose a λ as high as Ridge's (10^{1}), it would zero out almost all polynomial features, leaving a constant model with huge error. Therefore, Lasso is forced to pick much smaller λ values to retain the few available features.

In the middle range, the polynomial degree almost matches the complexity of the underlying Runge function. The model has enough capacity to capture the actual signal without being overwhelmed by noise or other variance. Here, all models reach their lowest validation error. However, Lasso is not capable of reaching the same minimum as Ridge does. The perfect value for λ is significantly lower than in the first area. Lasso's penalty terms are still lower than Ridge's (except for two spikes). Between a degree of 8 and 20, they reach their minimum at 10^{-8}, which is defined by the given list. The penalty terms for Ridge are shrinking more slowly but drop at a degree of 16 to the set minimum, rising to larger values afterwards to prevent overfitting effects. To compare the penalization effects of Ridge and Lasso polynomials of degree 17 can be taken. Here, Ridge keeps all features (x^{1} through x^{17}) active, because its tiny λ imposes almost no penalty. However, even with a small λ, Lasso's L_1 geometry still zeroes out truly redundant features, resulting in a sparse model with essentially the same optimal MSE. In this region, OLS works perfectly fine; the test error is comparable to the error with Ridge. OLS does not overfit strongly in this area. Therefore, adding penalty terms would introduce unnecessary bias and would hurt performance. Cross-validation recognizes this and turns off regularization by selecting the smallest λs allowed on the grid.

With a degree of 18, the test MSE starts to increase again. From here on, the model is overfitting the data. By fitting the noise, it misses the actual data as high-power polynomial terms (e.g. x^{16}, x^{20}, x^{30}) introduce extreme variance and strong multicollinearity. The test MSE of OLS quickly increases. Ridge and Lasso both stay stable and successfully prevent overfitting, keeping the test MSE low and flat around 10^{-2}. Regularization counteracts the extremely high variance introduced by overfitting of the high polynomial. However, Ridge and Lasso are handling the excess features differently. As shown in the figure, Ridge increases steadily from $\approx 10^{-7}$ up to 10^{-3}. Ridge shrinks all features, but doesn't set any feature to zero, to tame an increasing number of high variance features, cross validation has to increase λ to apply stronger overall suppression. In comparison, Lasso can set redundant features to zero. As the higher features are completely unnecessary, it simply sets their weights to zero. That is why it does not need to increase λ steadily - a small steady λ is sufficient to build a stable model.

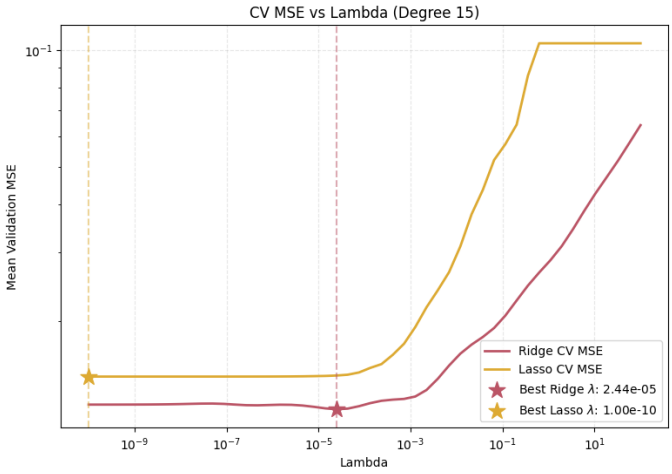


FIG. 30: For the degree 15 polynomial fit the CV MSE is printed according to the corresponding λ, comparing Ridge and Lasso.

Figure 30 presents this observation as well, for a degree of 15, Ridge can create a better result with a lower MSE and even with an increasing λ the error is not increasing as steep as the error for Lasso Regression. Because Lasso sets the features to exactly zero, a big λ eliminates key components of the true signal introducing high bias. In contrast, the error for Ridge increases much smoother as every feature gets shrunk evenly. Thus, it retains the predictive contributions of all features longer.

Figure 29 reveals which model is the best for certain use cases. OLS works perfectly fine, when there is no need to suppress features, so when polynomial degree matches the true complexity and signal. Then there is no need for hyperparameters, but when the degree increases

the error will explode without them. Ridge however uses hyperparameter to shrink all features smoothly to prevent overfitting. This works best when all features are highly correlated or evenly contribute to the outcome. This is why Ridge works better here as every feature above a degree of 15 is highly correlated and unnecessary so they can be shrunk effectively. Lasso struggles here, because of this. It tends to arbitrarily pick one feature to keep and zeroes out the rest, which can make its predictions slightly unstable across cross-validation folds so the resulting MSE is slightly above Ridge. In conclusion, OLS works perfectly fine when the model is not overfitting. In those areas it should be used preferably as it computes much faster and with a closed analytical form. Nevertheless, it always comes along with the risk of overfitting the data. When there is risk of overfitting, Ridge and Lasso work more effectively. Which model produces better results really depends on the data. When all features are highly correlated Ridge will produce better results. In contrast, Lasso outperforms Ridge when the features are not evenly contributing to the result, as it shrinks every unnecessary feature to zero. One example where Lasso outperforms Ridge by far can be found in the appendix (section E.

Figure 31 compares OLS, Ridge and Lasso Regression for different degrees, setting the focus to over - and underfitting. The diagram also plots the true Runge function. Clearly a degree 4 polynomial fails to fit the function completely, but even at this degree, an oscillation near the boundaries can be identified. This oscillation increases with the degree, at a degree of 8 three local extrema at each side of the global maximum can be detected. At this degree there is no need to introduce penalty terms yet. Nonetheless, a degree of 15 clearly demonstrates the need for penalty terms, as the OLS curve is oscillating near $x = 1.00$ with $y_{centered} \approx -1.5$. Also its maximum of the OLS curve shifted towards a bigger x. In the center, a few noise points around $x \approx 0.05$ lie above the true Runge curve, pulling the unregularized polynomial peak toward them.

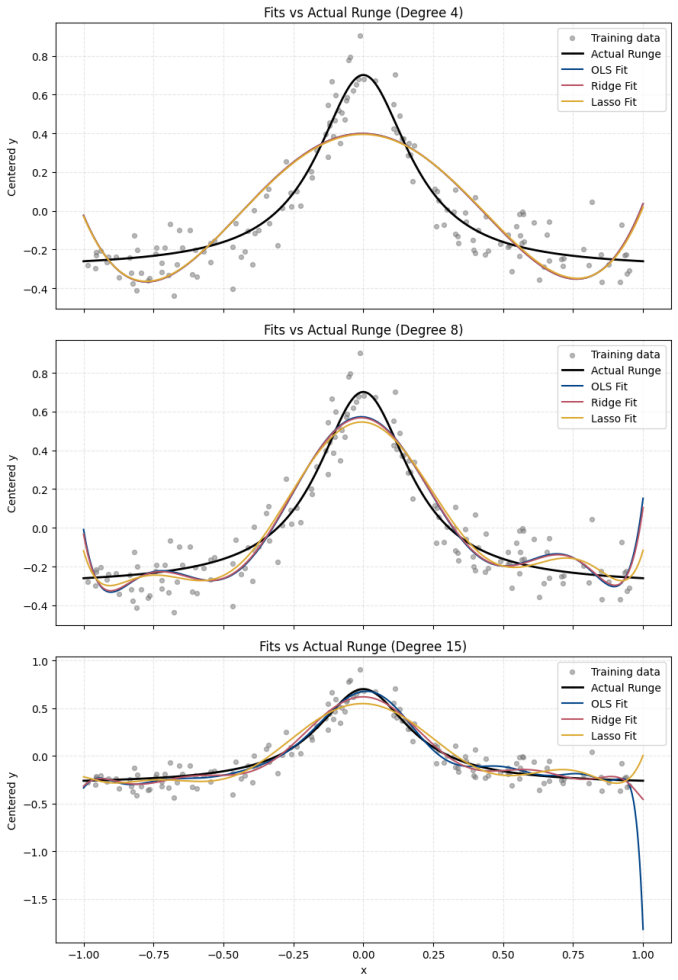


FIG. 31: Comparison of fits of different degrees, one considered good (degree = 15), one considered underfit and one considered overfit. For each scenario it compares Ridge, Lasso and OLS.

IV. CONCLUSION

We fitted noisy samples of Runge's function with OLS, Ridge and Lasso regression and solved these problems both in closed form and with gradient-based optimizers. Our results answer three questions: which model predicts best, which optimizer converges fastest, and which method is computationally most efficient.

Prediction quality. Ridge regression gave the best predictions overall: it matched OLS near the optimal polynomial degree and remained stable where OLS overfitted. For plain OLS, the bias dominates at low degrees, which cannot represent the sharp peak of the Runge function, while at high degrees the polynomial fits the noise and the variance dominates. Cross-validated OLS reached its lowest test error at degree 11 for $n = 150$, Ridge at degree 12 with $\lambda = 7.91 \cdot 10^{-6}$, and Lasso at degree 18 with $\lambda = 10^{-8}$, the smallest value on our grid, so that the optimal Lasso fit is close to an unpenalized one. Since the cross-validated error barely changes between degrees 11 and 18, these optima are consistent, and all three methods reach errors of order 10^{-2} there (fig. 29). The methods differ above degree 17: the OLS error grows rapidly, whereas Ridge and Lasso's errors remain at approximately 10^{-2}. Regularization therefore does not noticeably lower the minimal error for this problem, but it makes the predictions robust against an overly complex model. Larger samples allow higher degrees before overfitting sets in, with optimal degrees of 8, 15 and 24 for $n = 50$, 200 and 450 (section III.C), respectively. Lasso reached a slightly higher error than Ridge at degree 15 (fig. 30), as the strongly correlated polynomial

features make its feature selection unstable. On data where most features are irrelevant, however, its sparsity pays off: with 195 irrelevant features out of 200, Lasso reached an MSE of 1.98 compared with 120.82 for Ridge (section E).

Convergence speed. Which optimizer converged fastest depended on the conditioning of the problem and on how well the learning rate was known. With the analytically optimal learning rate γ^*, plain GD reached a precision of 10^{-6} in 3728 steps for OLS and in 198 steps for Ridge, and the ratio of the step counts matches the ratio of the condition numbers,

$$\frac{\kappa_{OLS}}{\kappa_{Ridge}} = 18.74 \approx 18.83 = \frac{\#\text{steps}_{OLS}}{\#\text{steps}_{Ridge}}. \tag{38}$$

For the well-conditioned Ridge problem, plain GD at γ^* was the fastest of all methods, ahead of Adam (257 steps) and momentum (272 steps). For the ill-conditioned OLS problem, momentum was fastest, with 280 to 285 steps for $\gamma \geq 0.9\,\gamma_{\max}$, about 13 times fewer than plain GD at γ^* (tables VII to XI). Without knowledge of γ^*, Adam is the most robust choice: it converged for every learning rate in $[10^{-3}, 10^{1}]$ within at most about 4×10^3 steps for OLS and 2×10^3 for Ridge, and needed the fewest steps for $\gamma \lesssim 10^{-1}$ (OLS) and $\gamma \lesssim 10^{-2}$ (Ridge). Plain GD, in contrast, diverged above $\gamma_{\max}$ and converged within 20 000 steps only for $\gamma \geq 5\times10^{-2}$ (OLS) and $\gamma \geq 2\times10^{-3}$ (Ridge), AdaGrad slowed down at small learning rates, and RMSprop never reached the tolerance but kept oscillating at a distance of the order of γ from the minimum (figs. 21 and 22)

Computational cost. For OLS and Ridge on a problem of this size, the closed-form solution is computationally the best method: a single SVD yields the exact minimum, whereas every iterative method needs hundreds to thousands of gradient evaluations and only approximates it. One analytical gradient costs $4np$ flops, i.e. 2000 flops for the degree-5 fit with $n = 100$, so even momentum GD needed about 6×10^5 flops to reach a precision of 10^{-6}. Automatic differentiation costs about two evaluations of the cost function per gradient, roughly 20% more than the analytical gradient, but makes deriving the gradient unnecessary, whereas central differences need $2p$ evaluations and are less accurate. For Lasso, which has no closed-form solution, the coordinate descent of scikit-learn was both faster and more precise than our subgradient implementation, which set no coefficient exactly to zero. Mini-batch SGD lowered the training error fastest at first, since each update costs only $\mathcal{O}(Mp)$ instead of $\mathcal{O}(np)$ operations (fig. 24), but it leveled off at an excess cost of about 10^{-2}. With $M = 25$, it reached 1.15×10^{-2} after 1000 epochs and 1.08×10^{-2} after 2000 epochs, i.e. 160 000 sample gradient evaluations, with a validation MSE of 0.0300. With the same number of evaluations, full-gradient Adam reached 3.86×10^{-3} and 0.0231, while the closed-form OLS solution has a validation MSE of 0.0204 (table XII). The cheap updates of SGD therefore pay off only for data sets large enough that full gradients and closed-form solutions become too expensive.

Limitations. Each of our main analyses used a single seed (2026) for the noise and the initial parameters, and we investigated only a few sample sizes and, apart from the noise scan for OLS in fig. 3, a fixed noise level. Apart from the spread across cross-validation folds, the reported MSE values, optimal degrees, penalty parameters and iteration counts therefore carry no error estimate. The flat test-error curves around the optimal degrees (sections III.A and III.C) suggest that the optimal degree in particular depends on the realization of the noise. In sections III.A and III.B, we chose the polynomial degree and λ with the lowest test MSE and reported that same test MSE. Since the smallest of many noisy estimates tends to come from a model that happened to fit the test set well, these errors underestimate the error on new data. The iteration counts of the gradient descent methods stem from a single random initialization, and we stopped all runs after 20 000 iterations. A method that did not reach the tolerance within this budget has not necessarily failed to converge: plain GD with $\gamma = 0.1\,\gamma_{\max}$ for OLS, for example, still converges linearly, only slowly (fig. 16). Finally, all optimization problems studied here are convex, so every local minimum is a global one. The main expected advantage of SGD, escaping poor local minima through its gradient noise, could therefore not show, and our comparison judges SGD by its cost and accuracy alone.

Outlook. Averaging over several seeds for the noise and the initial parameters would provide error estimates for all reported quantities and more reliable iteration counts, and a separate validation set or nested cross-validation would remove the optimistic bias of the selected models. A validation-based stopping rule for SGD and full-gradient Adam would detect convergence and avoid unnecessary gradient evaluations. Finally, applying the optimizers to non-convex cost functions, such as those of neural networks, would show whether the gradient noise of SGD helps to escape local minima and thereby outweighs its slower final convergence.

REFERENCES

A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind (2018), "Automatic Differentiation in Machine Learning: a Survey," Journal of Machine Learning Research **18** (153), 1–43.

J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang (2018), "JAX: composable transformations of Python+NumPy programs," .

J. Duchi, E. Hazan, and Y. Singer (2011), "Adaptive Subgradient Methods for Online Learning and Stochastic Opti-

mization," Journal of Machine Learning Research **12** (61), 2121–2159.

B. Efron (1979), "Bootstrap Methods: Another Look at the Jackknife," The Annals of Statistics **7** (1), 1–26.

C. R. Harris, *et al.* (2020), "Array Programming with NumPy," Nature **585**, 357–362.

G. Hinton (2012), "Neural Networks for Machine Learning," Lecture notes for Coursera: Neural Networks for Machine Learning, lecture 6e for 2012 Coursera course.

M. Hjorth-Jensen (2026), "Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning," Lecture notes for FYS-STK3155/4155, University of Oslo, course PDF dated 16 August 2026.

D. P. Kingma, and J. Ba (2015), "Adam: A Method for Stochastic Optimization," in *3rd International Conference on Learning Representations (ICLR)*.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay (2011), "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research **12**, 2825–2830.

C. Runge (1901), "Über empirische Funktionen und die Interpolation zwischen Äquidistanten Ordinaten," Zeitschrift für Mathematik und Physik **46**, 224–243.

R. Tibshirani (1996), "Regression Shrinkage and Selection via the Lasso," Journal of the Royal Statistical Society: Series B (Methodological) **58** (1), 267–288.

Appendix A: Table of abbreviations

TABLE V: Table of used abbreviations and acronyms.

Acronym	Meaning	Description
OLS	Ordinary Least Squares	Method to find optimal parameters
MSE	Mean Squared Error	The average error term squared
SVD	Singular Value Decomposition	Matrix factorization technique
GD	Gradient Descent	Optimization algorithm
SGD	Stochastic Gradient Descent	Optimization algorithm
CV	Cross Validation	Resampling method

Appendix B: Coefficients of the OLS model fitted to the Runge function

Note that the coefficients below are fitted on data, which was randomly sampled with additive noise and was furthermore centered and scaled as explained in section II.A.5. Plots of the individual coefficients θ can be found in the Github repository under: GitHub Repository – Plots of OLS Coefficients.

TABLE VI: Coefficient values for a polynomial fit of the Runge function with additional noise

Degree	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7	θ_8	θ_9	θ_{10}	θ_{11}	θ_{12}	θ_{13}	θ_{14}	θ_{15}
2	−0.0	−0.21													
3	0.03	−0.22	−0.03												
4	−0.02	−0.7	0.05	0.5											
5	−0.07	−0.7	0.23	0.49	−0.14										
6	−0.03	−1.27	0.08	2.18	−0.07	−1.15									
7	−0.08	−1.29	0.42	2.24	−0.74	−1.19	0.38								
8	−0.07	−2.12	0.29	6.73	−0.35	−8.59	0.13	3.77							
9	0.08	−2.17	−1.33	6.99	5.44	−9.02	−7.63	3.99	3.47						
10	0.08	−2.61	−1.35	10.84	5.31	−20.51	−7.23	17.73	3.2	−5.69					
11	−0.09	−2.53	1.26	10.25	−8.37	−18.85	23.4	15.63	−27.25	−4.73	11.07				
12	−0.12	−3.55	2.11	23.14	−13.59	−75.66	36.32	127.24	−41.4	−105.25	16.71	33.84			
13	0.04	−3.73	−1.45	25.67	15.21	−88.07	−66.65	154.36	137.13	−132.34	−130.92	43.88	46.68		
14	0.02	−4.29	−0.87	35.68	9.45	−153.77	−42.32	359.36	88.23	−457.77	−84.62	297.3	30.14	−76.74	
15	0.01	−4.29	−0.43	35.55	4.61	−152.81	−18.05	355.9	24.92	−451.27	3.88	291.3	−32.6	−74.6	17.69

Appendix C: Gradient descent optimizer

The following gives a tabular overview of the convergence of the different gradient descent optimizers each for different initial learning rates. Here $\gamma_{\max}$ and γ^* are the maximum and ideal initial learning rate of the plain gradient descent as given in eqs. (18) to (19) If the optimizer does not reach the tolerance of 10^{-6} after 20 000 steps, the method is listed as *did not converge*.

Method	Learning Rate	Steps to Convergence
OLS	$0.1000 * \gamma_{\max} = 0.0354$	did not converge
	$0.5000 * \gamma_{\max} = 0.1769$	7406
	$0.9000 * \gamma_{\max} = 0.3184$	4112
	$0.9900 * \gamma_{\max} = 0.3503$	3737
	$0.9981 * \gamma_{\max} = 0.3531$	3728
Ridge	$0.1000 * \gamma_{\max} = 0.0342$	1959
	$0.5000 * \gamma_{\max} = 0.1709$	386
	$0.9000 * \gamma_{\max} = 0.3075$	212
	$0.9900 * \gamma_{\max} = 0.3383$	662
	$0.9652 * \gamma_{\max} = 0.3298$	198

TABLE VII: Steps to convergence for different learning rates in plain gradient descent for OLS and Ridge regression.

Method	Learning Rate	Steps to Convergence
OLS	$0.1000 * \gamma_{\max} = 0.0354$	3576
	$0.5000 * \gamma_{\max} = 0.1769$	578
	$0.9000 * \gamma_{\max} = 0.3184$	280
	$0.9900 * \gamma_{\max} = 0.3503$	285
	$0.9981 * \gamma_{\max} = 0.3531$	285
Ridge	$0.1000 * \gamma_{\max} = 0.0342$	276
	$0.5000 * \gamma_{\max} = 0.1709$	272
	$0.9000 * \gamma_{\max} = 0.3075$	251
	$0.9900 * \gamma_{\max} = 0.3383$	269
	$0.9652 * \gamma_{\max} = 0.3298$	272

TABLE VIII: Steps to convergence for different learning rates in momentum gradient descent for OLS and Ridge regression.

Method	Learning Rate	Steps to Convergence
OLS	$0.1000 * \gamma_{\max} = 0.0354$	did not converge
	$0.5000 * \gamma_{\max} = 0.1769$	14827
	$0.9000 * \gamma_{\max} = 0.3184$	7450
	$0.9900 * \gamma_{\max} = 0.3503$	6843
	$0.9981 * \gamma_{\max} = 0.3531$	6798
Ridge	$0.1000 * \gamma_{\max} = 0.0342$	19261
	$0.5000 * \gamma_{\max} = 0.1709$	1135
	$0.9000 * \gamma_{\max} = 0.3075$	477
	$0.9900 * \gamma_{\max} = 0.3383$	424
	$0.9652 * \gamma_{\max} = 0.3298$	437

TABLE IX: Steps to convergence for different learning rates in adagrad gradient descent for OLS and Ridge regression.

Method	Learning Rate	Steps to Convergence
OLS	$0.1000 * \gamma_{\max} = 0.0354$	did not converge
	$0.5000 * \gamma_{\max} = 0.1769$	did not converge
	$0.9000 * \gamma_{\max} = 0.3184$	did not converge
	$0.9900 * \gamma_{\max} = 0.3503$	did not converge
	$0.9981 * \gamma_{\max} = 0.3531$	did not converge
Ridge	$0.1000 * \gamma_{\max} = 0.0342$	did not converge
	$0.5000 * \gamma_{\max} = 0.1709$	did not converge
	$0.9000 * \gamma_{\max} = 0.3075$	did not converge
	$0.9900 * \gamma_{\max} = 0.3383$	did not converge
	$0.9652 * \gamma_{\max} = 0.3298$	did not converge

TABLE X: Steps to convergence for different learning rates in rmsprop gradient descent for OLS and Ridge regression.

Method	Learning Rate	Steps to Convergence
OLS	$0.1000 * \gamma_{\max} = 0.0354$	2518
	$0.5000 * \gamma_{\max} = 0.1769$	925
	$0.9000 * \gamma_{\max} = 0.3184$	648
	$0.9900 * \gamma_{\max} = 0.3503$	611
	$0.9981 * \gamma_{\max} = 0.3531$	608
Ridge	$0.1000 * \gamma_{\max} = 0.0342$	419
	$0.5000 * \gamma_{\max} = 0.1709$	262
	$0.9000 * \gamma_{\max} = 0.3075$	263
	$0.9900 * \gamma_{\max} = 0.3383$	268
	$0.9652 * \gamma_{\max} = 0.3298$	257

TABLE XI: Steps to convergence for different learning rates in adam gradient descent for OLS and Ridge regression.

Appendix D: Numerical Values for each epoch

Epochs	**Optimization Method**	**Final CV Train Excess**	**Final CV Val MSE**
100	Minibatch SGD ($M = 25$)	1.573×10^{-2}	0.0319
	Full-Grad Adam ($M = 80$)	1.563×10^{-2}	0.0316
500	Minibatch SGD ($M = 25$)	1.228×10^{-2}	0.0312
	Full-Grad Adam ($M = 80$)	9.662×10^{-3}	0.0283
2000	Minibatch SGD ($M = 25$)	1.082×10^{-2}	0.0300
	Full-Grad Adam ($M = 80$)	3.862×10^{-3}	0.0231
-	**Analytical OLS (Exact)**	**0.000**	**0.0204**

TABLE XII: Comparison of training excess cost and validation MSE across epochs, highlighting the variance floor of mini-batch SGD compared to Full-Gradient Adam and the exact analytical OLS baseline.

Appendix E: Example: Lasso outperforms Ridge Regression

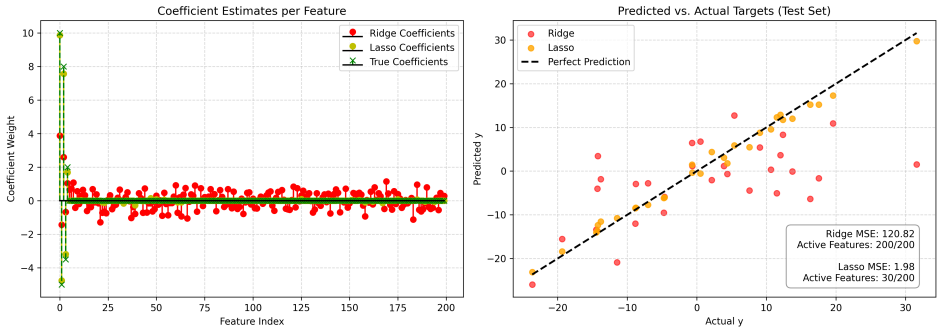


FIG. 32: This figure compares Ridge and Lasso Regression on a dataset where 195 out of 200 features are unnecessary.

To demonstrate the power of Lasso Regression, this example (figure 32) creates data with 195 completely unnecessary features. As Ridge Regression can't set them to zero, it fails here to create a good fit. This is why its MSE is almost 60 times bigger than the MSE from the Lasso Regression.

Appendix F: AI Declaration

The following points describe, where Artificial Intelligence methods were used to alter or write parts of this paper. For every instance of Large Language Model usage the tool, access method, usage level and a short description of what was done will be provided.

- Section: Abstract; Tool: GPT-6-Astra; Access method: Desktop App; Usage level: 1; Date: 2.10.2026; Description: spell and grammar checking, fixes reviewed and manually implemented by user.
- Section: Introduction; Tool: GPT-6-Astra; Access method: Desktop App; Usage level: 1; Date: 2.10.2026; Description: spell and grammar checking, fixes reviewed and manually implemented by user.
- Section: Methods; Tool: Claude Opus 5.5; Access method: claude.ai; Usage level: 2; Date: 28/09/2026; Description: Prompted Claude to do spell- and grammar-check and suggest improvements in section II.E.4 and section II.E.5. Claude improved grammar and language and suggested clarification of the momentum term in Adam. Claude was prompted to implement its suggestion. And more parameters were added in section II.E.5 for clarification by the user. The text was revised and checked.
- Section: Methods; Tool: GPT-6-Astra; Access method: Desktop App; Usage level: 1; Date: 2.10.2026; Description: spell and grammar checking, fixes reviewed and manually implemented by user.
- Section: Methods; Tool: Claude Opus 5.5; Access method: claude.ai; Usage level: 3; Date: 28/09/2026; Description: Prompted Claude to generate the pseudocode for plain gradient descent. Pseudo Code was revised and checked.
- Section: Methods; Tool: Claude Opus 5.5; Access method: claude.ai; Usage level: 3; Date: 05/10/2026; Description: Claude checked and noticed, that the relative cost of AD to analytical gradient is missing. Claude was prompted to count and write a paragraph in the same style as before. The output was checked and revised by the user.
- Section: Results; Tool: Claude Opus 5.5; Access method: claude.ai; Usage level: 2; Date: 28/09/2026; Description: Prompted Claude to do spell- and grammar-check and suggest improvements in section III.D.1. Claude suggested clarification regarding the convergence behavior of RMSprop in fig. 22. The suggested change was implemented. The text was revised, checked and altered by the user.
- Section: Results; Tool: GPT-6-Astra; Access method: Desktop App; Usage level: 1; Date: 2.10.2026; Description: spell and grammar checking, fixes reviewed and manually implemented by user.
- Section: Results; Tool: GPT-6-Luna; Access method: Desktop App; Usage level: 2; Date: 5.10.2026; Description: Small section clarification / rephrasing of sentence in SGD result analysis. Manually checked and implemented by user.
- Section: Conclusion; Tool: GPT-6-Astra; Access method: Desktop App; Usage level: 1; Date: 2.10.2026; Description: spell and grammar checking, fixes reviewed and manually implemented by user.
- Section: Conclusion; Tool: Claude Opus 5.5; Access method: claude.ai; Usage level: 3; Date: 5.10.2026; Description: Claude was prompted to generate the limitation paragraph in the same writing style as the report, based on bullet points of limitations given by the user. The text was reviewed and altered.
- Section: Conclusion; Tool: Claude Opus 5.5; Access method: claude.ai; Usage level: 2; Date: 5.10.2026; Description: Claude restructured the conclusion.

A short overview of artificial intelligence usage for code generation is provided below. For a more detailed account of the usage, please refer to the code under: GitHub Repository – Project 1. All declared usage was in the file "Project1.ipynb" unless otherwise specified.

a Tool: GPT-5.6. - Luna; Access method: VSCode plugin; Usage Level: 2; Description: Used to help complete short plotting functions; Date: 23.9.2026

b Tool: GPT-5.6. - Luna; Access method: VSCode plugin; Usage Level: 2; Description: Used to help write short plotting functions; Date: 23.9.2026

c Tool: GPT-6-Astra; Access method: VSCode plugin; Usage Level: 2-3; Description: Used for plotting and debugging; Date: 23.9.2026, 2.10.2026

d Tool: GPT-6-Astra; Access method: VSCode plugin; Usage Level: 2-3; Description: Used for plotting and debugging; Date: 23.9.2026, 2.10.2026

e Tool: Google Gemini Flash; Access method: Cloud-based usage; Usage Level: 2-3; Description: Fixing the code output to accommodate latex format; Date: 1.10.2026

Tool: Copilot Inline Suggestion; Access method: VSCode plugin; Usage Level: 1; description: Faster writing with inline auto-completion; Date: September 2026

f Tool: Copilot Inline Suggestion; Access method: VSCode plugin; Usage Level: 1; description: Faster writing with inline auto-completion; Date: September 2026

g Tool: Google Gemini Pro 2.0; Access method: Cloud based usage; Usage Level: 2; description: Used for plotting / debugging / implementing tables in latex from terminal output; Date: 21.09.

Files where AI was used: fitting.py; jax_ad.py; kfold.py

h Tool: GPT-5.6 - Luna; Access method: VSCode plugin; Usage Level: 2; Description: Debugging the scripts that were build from the original jupyter notebooks, adding more detailed comments to the code sections / Helping to organize the notebook in separate scripts, providing logical outline; 28.09

Tool: Gemini Flash; Access method: cloud-based; Usage Level: 1; description: translated the first paragraph of section III.F.4 from german to english; Date 04.10

Files where AI was used: excesscost_cv_validation.py; minibatch_comparison.py

i Tool: Google Gemini Flash; Access method: Cloud based usage; Usage Level: 2; description: Fixing the latex code; Date: 01.10.

Tool: GPT-5.6 - Luna; Access method: VSCode plugin; Usage Level: 2; description: debugging and commenting the code; Date 25.09

Files where AI was used: kfold_sklearn.py; function_vs_degree.py

Tool: Gemini Pro 2.0; Access method: cloud-based; Usage Level 3; description: Created an example where Lasso Regression works better than Ridge, Gemini created code and plots for a problem idea we described to the model; Date 01.10

Files where AI was used: lassobetterthanridge.py